

Ferngesteuertes Fahrzeug mit Positionserkennung

Verfasser:	David Kedra
	Berdan Kilic
Betreuer:	Dipl.-Ing. Ahmet Kilic
Zusatzbetreuer:	Dipl.-Ing. Heinrich Haager
Abteilung:	Elektronik und technische Informatik
Schuljahr:	2023/24
Klasse:	5AHEL

htl
bildung mit zukunft

htl donaustadt
Donaustadtstraße 45
1220 Wien

Abteilung: Elektronik und technische Informatik
Verfasser: David Kedra und Berdan Kilic

VERFASSERERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Wien, 1. Januar 2024

Verfasser / innen:

David Kedra

Unterschrift

Berdan Kilic

Unterschrift

PROBLEMSTELLUNG

Präzise Fahrzeugortung und -navigation sind in der Technologie von großer Bedeutung. Das Ziel des Projekts ist die Entwicklung eines ferngesteuerten Fahrzeugs, wobei die Fernsteuerung, die mit einem Bildschirm ausgestattet ist, der die Umgebung und Änderungen in der Umgebung speichern und anzeigen kann. Das Ziel ist die Realisierung einer kostengünstigen Lösung für die einfache Fernsteuerung und die genaue Umgebungsmessung mit Navigationshilfe.

Mögliche Anwendungen umfassen die Überwachung von groben Veränderungen eines Raumes, die Navigationshilfe durch eine optimale Routenfindung zu einer markierten Position unter Vermeidung von Hindernissen sowie die Ortung von Räumen, die von Menschen nicht betreten werden können.

KURZFASSUNG

Bei dem Projekt handelt es sich um ein Fahrzeug, das über eine Fernbedienung gesteuert wird. Die Fernbedienung ist mit einem Display ausgestattet, das die aktuelle Umgebung des Fahrzeugs anzeigen kann. Ziel ist es, eine Steuerung zu realisieren, die sowohl eine präzise als auch eine benutzerfreundliche Navigation ermöglicht. Das Fahrzeug ist mit der Funktion ausgestattet, die Umgebung zu speichern, auch wenn sich die Position von Objekten im Raum verändert. Des Weiteren ist die Anzeige einer optimalen Route von der aktuellen Position zu einer markierten Position möglich, um eine Navigationsunterstützung zurück zu dieser Position zu bieten.

Das System besteht aus zwei Hauptkomponenten: dem Fahrzeug und dem Controller, beide ausgestattet mit einem ESP32 Mikrocontroller. Das Fahrzeug ist mit vier DC-Motoren für die Fortbewegung und einem 360° Entfernungssensor zur Umgebungserfassung ausgerüstet. Die manuelle Steuerung erfolgt über einen externen Controller, der mit einem Joystick und einem Display ausgestattet ist, um die Steuerbefehle zu übermitteln und die vom Fahrzeug gesammelten Umgebungsdaten visuell darzustellen. Die Kommunikation zwischen Fahrzeug und Controller erfolgt über das eingebaute Bluetooth-Modul im ESP32.

Das Ergebnis des Projekts zeigt, wie effektiv die Kombination aus fortschrittlichen Mikrocontrollern, Sensorik und drahtloser Kommunikation ein ferngesteuertes Fahrzeugsystem verbessern kann. Der Prototyp erfüllt alle gesetzten Ziele, indem er eine präzise Navigation und eine intuitive Steuerung bietet, die durch Echtzeitdaten der Umgebung unterstützt werden.

ABSTRACT

The project involves a vehicle that is controlled via a remote control. The remote control is equipped with a display that can show the vehicle's current surroundings. The aim is to realise a control system that enables both precise and user-friendly navigation. The vehicle is equipped with the function of memorising the surroundings, even if the position of objects in the room changes. It is also possible to display an optimal route from the current position to a marked position to help navigate back to that position.

The system consists of two main components: the vehicle and the controller, both equipped with an ESP32 microcontroller. The vehicle is powered by four DC motors for movement and a 360° distance sensor for environmental detection. Manual control is provided by an external controller equipped with a joystick and a display to transmit control commands and visualise the environmental data collected by the vehicle’s sensor. Communication between the vehicle and the controller takes place via the built-in Bluetooth module in each ESP32.

The result of the project shows how effectively the combination of advanced microcontrollers, sensor technology and wireless communication can improve a remote-controlled vehicle system. The prototype fulfils all set goals by providing precise navigation and intuitive control supported by real-time data of the environment.

INHALTSVERZEICHNIS

1 Danksagung 8

2 Projektplanung 9

2.1 David Kedra 9
2.2 Berdan Kilic 10

3 Grundlagen und Methoden 10

3.1 Blockschaltbild 11
3.2 Schaubild der Fernsteuerung und geplante Visualisierung 12
3.3 Hardware 13
3.3.1 Fernsteuerung [David Kedra] 13
3.3.2 Fahrzeug [Berdan Kilic] 19

4 Durchführung 25

4.1 Bluetooth-Kommunikation [David Kedra] 25
4.1.1 Ablauf 25
4.1.2 Datenformatierung und -übertragung 28
4.1.3 Bluetooth-Kopplung 30
4.1.4 Algorithmus für das Empfangen 33
4.1.5 Algorithmus für das Senden 37
4.2 Umgebungsberechnung und -speicherung [David Kedra] 38
4.2.1 Einleitung 38
4.2.2 Klassenbeschreibung 39
4.2.3 Datenverarbeitung 40
4.2.4 Optimierung der Umgebungsspeicherung 41
4.3 Routenfindung [David Kedra] 45
4.4 Darstellung auf dem Bildschirm [David Kedra] 48
4.4.1 Initialisierung und Klassenbeschreibung 48
4.4.2 Darstellungsfunktionen 49
4.4.3 Routendarstellung 49
4.5 Motorensteuerung [Berdan KILIC] 50
4.5.1 L298N-Treiber 50
4.5.2 Code 53
4.6 Datenerfassung vom Lidar-Sensor [Berdan KILIC] 58
4.6.1 Sensormessungen 58

4.6.2 Sensordaten 59
4.6.3 Code 60
4.7 Hardwareintegration im Fahrzeug [Berdan KILIC] 64
4.7.1 Integration des L298N-Treibers 64
4.7.2 Anschluss eines Gleichstrommotors 65
4.7.3 Versorgung 66

5 Ergebnisse 67

5.1 Platine der Fernsteuerung [David Kedra] 67
5.1.1 Stromlaufplan 67
5.1.2 Druckvorlage mit GND-Fläche 68
5.1.3 Druckvorlage ohne GND-Fläche 68
5.1.4 Bestückungsplan Top 69
5.1.5 Bestückungsplan Bottom 69
5.1.6 Leiterplattenfertigungszeichnung 70
5.1.7 I/O-Belegung des ESP32 der Fernsteuerung [David Kedra] 71
5.1.8 Stückliste (Fernsteuerung) [David Kedra] 71
5.2 Platine des Fahrzeugs [Berdan KILIC] 72
5.2.1 Stromlaufplan 72
5.2.2 Druckvorlage mit GND-Fläche 73
5.2.3 Druckvorlage ohne GND-Fläche 73
5.2.4 Bestückungsplan (Top) 74
5.2.5 Bestückungsplan (Bottom) 74
5.2.6 Leiterplattenfertigungszeichnung 75
5.2.7 I/O-Belegung des ESP32 des Fahrzeugs 76
5.2.8 Stückliste (Fahrzeug) 76
5.3 Flussdiagramm Fernsteuerung [David Kedra] 77
5.4 Flussdiagramm Fahrzeug [Berdan KILIC] 79

6 Literatur- und Quellenverzeichnis 82

7 Abbildungsverzeichnis 84

8 Tabellenverzeichnis 86

9 Verwendete Abkürzungen 86

10 Anhang 87

10.1 Stückliste 87
10.2 Kostendarstellung 87
10.3 Bedienungsanleitung 88

1 Danksagung

Wir möchten uns ganz herzlich bei unserem Betreuer Prof. DI Ahmet Kilic bedanken, der uns bei allen Fragen unterstützt und uns bei der Umsetzung unserer Ideen geholfen hat. Ein großes Dankeschön geht auch an unseren Zusatzbetreuer Prof. DI Heinrich Haager, der uns mit seinem Fachwissen bei der Software und beim Brainstorming von Ideen sehr unterstützt hat. Wir möchten uns außerdem bei Prof. Ing. Paul Necas bedanken, der uns als zusätzlicher Laborlehrer bei einigen Problemen und Fragen geholfen hat.

2 Projektplanung

2.1 David Kedra

Arbeitspunkte	Beschreibung der Arbeitspunkte	Zielsetzung
Bluetooth Kommunikation	Projektspezifische, bidirektionale Bluetooth-Kommunikation zwischen Fahrzeug und Fernsteuerung.	Ermöglichen der drahtlosen Kommunikation z.B. zur Fernsteuerung des Fahrzeugs und zum Senden von Sensordaten vom Fahrzeug zur Fernsteuerung
Bildschirm der Fernsteuerung	Anschließen und Testen des Displays an der Fernsteuerung.	Integration eines Bildschirms in der Fernsteuerung für die Anzeige von Information
Umgebungsspeicherung	Erfasste Daten des Entfernungssensors und Änderungen in der Umgebung speichern	Erfassen und Speichern relevanter Umgebungsdaten zur späteren Nutzung
Bildschirmanzeige der Umgebung	Anzeige der aktuellen und der gespeicherten Umgebung auf dem Bildschirm der Fernsteuerung.	Visualisierung der Umgebungsinformationen für den Benutzer auf dem Bildschirm der Fernsteuerung
Positionsauswahl und Routenfindung	Funktion zum Markieren einer Position mittels Joysticks auf dem Display. Berechnung der optimalen Route von der aktuellen Position zur markierten Position	Auswahl einer Position und Berechnung einer effizienten Route zu dieser Position für das Fahrzeug
Bildschirmanzeige der Route	Anzeige der berechneten optimalen Route von der aktuellen Position zur markierten Position auf dem Bildschirm.	Visualisierung der berechneten Route für den Benutzer auf dem Bildschirm der Fernsteuerung.
Platine Fernsteuerung	Vollständige Platine der Fernsteuerung: Alle Komponenten der Fernsteuerung in einer Platine integriert	Realisierung der Fernsteuerung als Platine

Tabelle 1: Arbeitspakete David Kedra

2.2 Berdan Kilic

Arbeitspunkte	Beschreibung der Arbeitspunkte	Zielsetzung
L298N-Treiber	Treiber einbauen und in die Motorenschaltung integrieren	Motorensteuerung ermöglichen
Motorensteuerung (Richtung)	Motoren durch Joystickwerte bezüglich Richtung kontrollieren	Ermöglichen der Steuerung der Fahrtrichtung des Fahrzeugs mit Joystick
Motorensteuerung (Geschwindigkeit)	Motoren durch Joystickwerte bezüglich Geschwindigkeit kontrollieren	Ermöglichen der Steuerung der Geschwindigkeit des Fahrzeugs mit Joystick
Lidar Sensor	Daten extrahieren laut Datenblatt für weitere Verarbeitung	Vom Sensor gesendete Datenpakete auswerten
Sensordaten verarbeiten	Datenpakete extrahieren	Winkel und Distanz aufnehmen und senden für Umgebungsanzeige
Platine Fahrzeug	Alle Komponenten der Fernsteuerung in einer Platine integriert	Realisierung der Fahrzeugtechnik als Platine

Tabelle 2: Arbeitspakete Berdan Kilic

3 Grundlagen und Methoden

3.1 Blockschaltbild

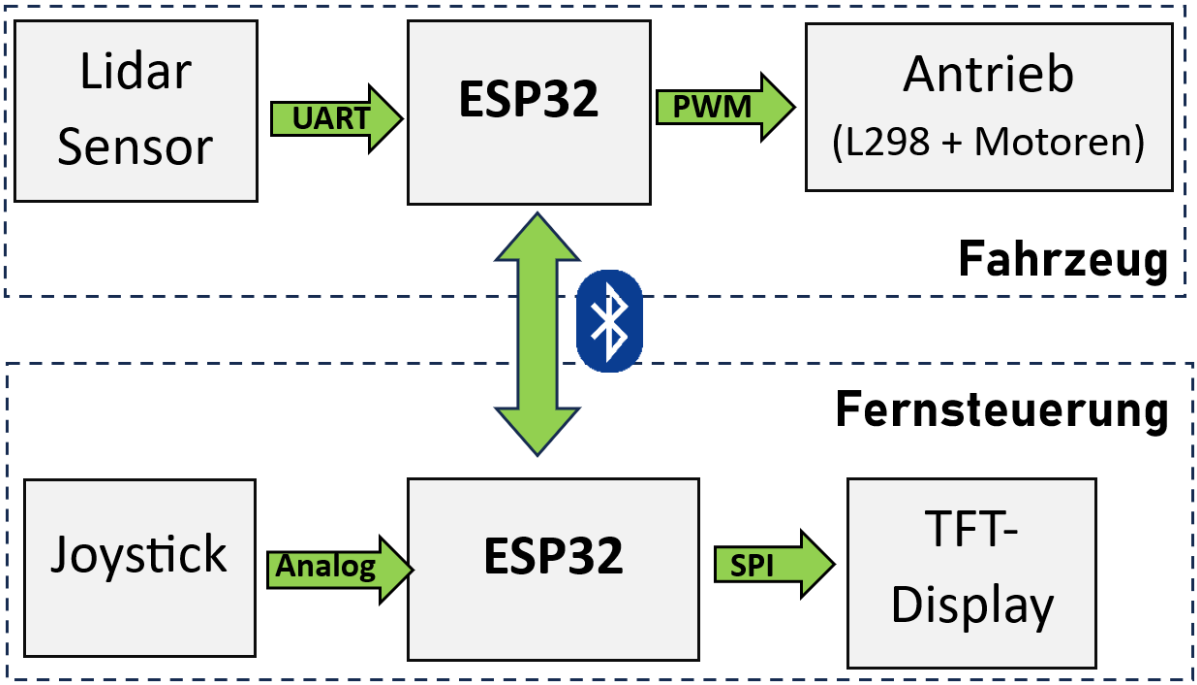


Abbildung 1: Blockschaltbild

3.2 Schaubild der Fernsteuerung und geplante Visualisierung



Abbildung 2: Fernsteuerung Schaubild

Das Display ist oben positioniert. Links am Rand unter dem Display befindet sich ein Ein- und Ausschalter. Rechts unten ist ein Joystick. Links vom Joystick ist ein Taster.

Der Mikrocontroller und die Stromversorgung befinden sich an der Unterseite der Platine.

Geplante Visualisierung:

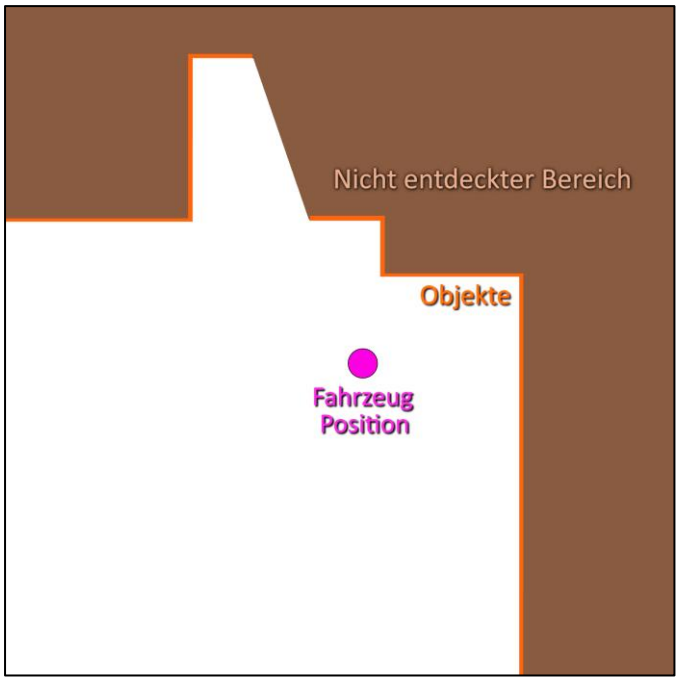


Abbildung 3: am Bildschirm geplante Visualisierung

3.3 Hardware

3.3.1 Fernsteuerung [David Kedra]

Mikrocontroller

ESP32-WROOM-32



Abbildung 4: ESP32-WROOM-32 Fernsteuerung, (ebay, 2024)

Der ESP32 wurde als Mikrocontroller für die Fernsteuerung gewählt, da er im Vergleich zu einem Arduino Leistungsstärker ist und ein Arduino für das Projekt nicht die erforderliche Leistung und Speicherkapazität verfügt. Der ESP32 verfügt über ausreichend Leistung und RAM, sodass ein Raspberry Pi nicht erforderlich ist. Ebenso verfügt der ESP32 über eine SPI-Schnittstelle, eine UART-Schnittstelle sowie ein integriertes Bluetooth-Modul, welche für dieses Projekt geeignet sind.

Bildschirm



Abbildung 5: Bildschirm, (lcdwiki, 2024)

Name	Parameter
Display Color	RGB 65K color
Screen Size	3.5 inch
Type	TFT
Driver IC	ILI9488
Resolution	480*320 Pixel
Module Interface	4-wire SPI interface
Active Area	48.96x73.44 mm
Module PCB Size	56.34x98 mm
Operating Temperature	-20℃~60℃
Storage Temperature	-30℃~70℃
Operating Voltage	3.3V~5V
Logic IO port voltage	3.3V
Weight	57 g

Tabelle 3: Bildschirm Parameter, (lcdwiki, 2024)

Number	Pin Label	Description
1	VCC	5V~3.3V power supply
2	GND	Ground
3	CS	LCD chip select signal, low level enable
4	RESET	LCD reset control signal, low level reset
5	DC/RS	LCD register / data selection signal, high level: register, low level: data
6	SDI(MOSI)	SPI bus write data signal
7	SCK	SPI bus clock signal
8	LED	Backlight control, high level lighting, if not controlled, connect 3.3V
9	SDO(MISO)	SPI bus read data signal, if you do not need the read function, you can not connect it

Tabelle 4: Bildschirm Schnittstellen, (lcdwiki, 2024)

Die Anzeige der Fernsteuerung besteht aus einem 3,5 Zoll Thin Film Transistor (TFT) Bildschirm mit einer Auflösung von 480 x 320 Pixeln und einer RGB 65k-Farbpalette, der über eine Serial Peripheral Interface (SPI) Schnittstelle durch den Treiber ILI9488 angesteuert wird.

Die SPI-Schnittstelle ermöglicht eine schnelle und bidirektionale Kommunikation zwischen einem Server-Gerät (in diesem Fall dem ESP32 Mikrocontroller) und einem Client-Gerät (dem SPI-Bildschirm) über eine minimale Anzahl von Leitungen.

Bei der Verbindung zwischen einem Mikrocontroller und einem SPI-Gerät werden mindestens vier Leitungen verwendet:

- **MOSI** (Master Out Slave In) bzw. **SDI** (Serial Data Input): Diese Leitung wird vom Mikrocontroller verwendet, um Daten an den Bildschirm zu senden.
- **MISO** (Master In Slave Out) bzw. **SDO** (Serial Data Output): Diese Leitung wird vom Bildschirm verwendet, um Daten an den Mikrocontroller zu senden.
- **SCK** (Serial Clock): Diese Leitung wird verwendet, um den Takt für die Datenübertragung zwischen Mikrocontroller und Bildschirm zu liefern. Der Takt auf der synchronisiert die Datenübertragung zwischen den beiden Geräten.

- **SS** (Slave Select) bzw. **CS** (Chip Select): Diese Leitung wird vom Mikrocontroller verwendet, um den Bildschirm als „Client“ auszuwählen und mit dem zu kommunizieren. Wenn mehrere Client-Geräte an denselben Mikrocontroller angeschlossen sind, wird diese Leitung verwendet, um zwischen ihnen zu wählen.

Zum Kommunizieren zwischen einem ESP32 Mikrocontroller und dem SPI-Bildschirm werden zwei weitere Leitungen benötigt:

- **RESET**: Diese Leitung dient zum Zurücksetzen des Bildschirms.
- **DC/RS** (Data Command / Register Select): Diese Leitung dient zur Auswahl, ob aktuell Daten oder Befehle an den Bildschirm gesendet werden.

Die Kommunikation zwischen Mikrocontroller und Bildschirm erfolgt in einem synchronisierten Modus, bei dem der Mikrocontroller den Takt vorgibt. Der Mikrocontroller sendet Befehle und Daten an den Bildschirm über die MOSI-Leitung, während der Bildschirm antwortet über die MISO-Leitung. Da der Bildschirm als Ausgabegerät nichts ausgibt, ist es in diesem Zustand nicht notwendig, ihn anzuschließen.

Der verwendete Bildschirm ist jedoch ein resistiver Touchscreen, der Berührungen erkennen und deren Position bestimmen kann. Um diese Informationen zu erhalten, ist die MOSI-Leitung notwendig. Da in dieser Diplomarbeit der Touchscreen nicht verwendet wird, wird darauf nicht weiter eingegangen. Zusätzlich ist an dem Bildschirm ein SD-Kartensteckplatz integriert, welcher jedoch ebenfalls nicht genutzt wird.

Der Treiber ILI9488 wurde für die Ansteuerung von TFT-Bildschirmen entwickelt worden. Die Ansteuerung erfolgt über die C++ Bibliothek „ILI9486_SPI“. Diese unterstützt ebenfalls den Treiber „ILI9488“ und ist im Arduino Library Manager auffindbar.

Joystick

COM-09032



Abbildung 6: Joystick, (digikey, 2024)

Der Joystick „COM-09032“ besteht aus zwei Achsen, x und y, und für jede Achse gibt es jeweils ein Potentiometer. Die Verbindung zwischen dem Joystick und dem ESP32-Mikrocontroller erfolgt über eine analoge Schnittstelle.

Bei einer Bewegung des Joysticks ändert sich der Widerstandswert an den Potenziometern, was eine Änderung der Spannung verursacht, die vom Mikrocontroller gelesen wird.

Der COM-09032 Joystick hat fünf Anschlusspins:

• **VCC**: Verbindung zur Versorgungspannung (3,3V). Hierbei ist es wichtig, die Versorgungsspannung entsprechend der IO-Spannung des Mikrocontrollers anzuschließen. Bei einem ESP32 wäre dies eine Spannung von 3,3V, im Gegensatz zu einem Arduino mit 5V.

• **GND**: Verbindung zur Masse (0V).

• **X-Achse**: Analoger Ausgang für die X-Achse.

• **Y-Achse**: Analoger Ausgang für die Y-Achse.

• **SW**: Ein Taster, der geschlossen wird, wenn der Joystick betätigt wird.

Bluetooth Modul (in finaler Version nicht vorhanden)

HC-05

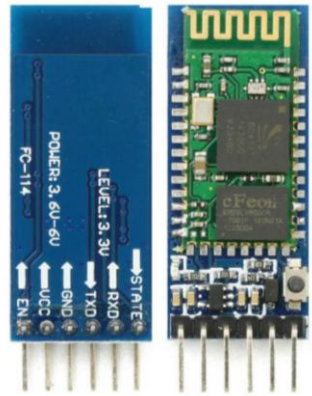


Abbildung 7: HC-05, (thinkrobotics, 2024)

Das HC-05 Modul ist mit einer UART-Schnittstelle ausgestattet, was die Kommunikation mit einem Mikrocontroller vereinfacht, da nur 2 Leitungen benötigt werden.

Das Modul verfügt über folgende Anschlüsse:

- **EN** (Eingang): Um das Modul für die Konfiguration zu aktivieren muss dieser Pin mit 3,3V versorgt werden.
- **VCC** (Eingang): Hier wird die Versorgungsspannung von 3,6 bis 6V, üblicherweise 5V, zugeführt.
- **GND** (Eingang): Dieser Pin ist mit der Masse (0V) verbunden.
- **TXD** (Transfer Data) (Ausgang): Serielle Schnittstelle zum Senden von Daten. Sie wird mit dem RXD-Pin des Mikrocontrollers verbunden.
- **RXD** (Receive Data) (Eingang): Serielle Schnittstelle zum Empfangen von Daten. Sie wird mit dem TXD-Pin des Mikrocontrollers verbunden.
- **STATE** (Ausgang): Dieser Pin gibt 3,3 Volt aus, wenn das Modul mit einem anderen Bluetooth Gerät gekoppelt ist, andernfalls 0 Volt.

Die UART-Schnittstelle ermöglicht eine serielle Datenübertragung zwischen zwei Systemen über TxD und RxD Anschlüsse, wobei ein gemeinsamer Masseanschluss wichtig ist. Die Baudrate muss identisch sein, und Daten werden asynchron übertragen, was bedeutet, dass sie ohne ein gemeinsames Taktsignal gesendet werden. Dies ermöglicht eine einfachere Implementierung und Kommunikation zwischen verschiedenen Systemen. (mathe-mit-methode, 2024)

3.3.2 Fahrzeug [Berdan Kilic]

Mikrocontroller: ESP32-WROOM-32



Abbildung 8: ESP32 für Fahrzeug
(ebay, 2024)

Der ESP32 wurde als Mikrocontroller auch für das Fahrzeug gewählt, da er im Vergleich zu einem Arduino leistungsstärker ist und ein Arduino für das Projekt nicht die erforderliche Leistung und Speicherkapazität verfügt. Der ESP32 verfügt über ausreichend Leistung und RAM, eine SPI-Schnittstelle, eine UART-Schnittstelle sowie ein integriertes Bluetooth-Modul, welche für dieses Projekt geeignet sind. Die Motorensteuerung, das Senden der Sensordaten und das Empfangen der Joystickwerte ist für den ESP32 problemlos umsetzbar. Es ist optimal, dass im Fahrzeug und im Controller der ESP32 verwendet wird, da die Kommunikation zwischen den beiden Mikrocontrollern essenziell und somit gut verwirklichbar ist.

Treiber: L298-Chip

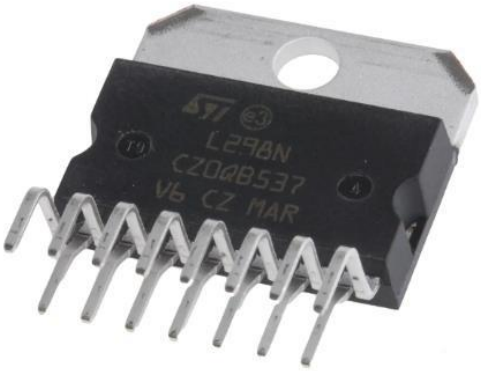


Abbildung 9: L298N-Chip (Stadler, 2023)

PIN	Bezeichnung	Erklärung
1,15	Sense A, Sense B	GND
2,3	OUT1, OUT2	Ausgänge der ersten H-Brücke
4	V_s	Spannungsversorgung der digitalen Steuerung (5V)
5,7	IN1, IN2	TTL-Eingänge für die erste H-Brücke
6,11	EnableA, EnableB	TTL-Eingänge zum Ein-/Ausschalten der Brücken
8	GND	Masse für den Motor und die Digitalsignale
10,12	IN3, IN4	TTL-Eingänge für die zweite H-Brücke
13,14	OUT3, OUT4	Ausgänge der zweiten H-Brücke

Tabelle 5: PIN-Tabelle für L298N (Stadler, 2023)

Um die 4 Schrittmotoren ansteuern zu können, werden Treiber benötigt. Ein L298N-Treiber kann 2 Schrittmotoren unabhängig voneinander steuern bzw. die Drehrichtung und Geschwindigkeit kontrollieren.

Antrieb des Fahrzeugs: 4 Schrittmotoren mit 4 Rädern

Für das Fahrzeug wurden 4 Schrittmotoren verwendet, welche in Abbildung 10 zu sehen sind.



Abbildung 10: Verwendeter Schrittmotor (getgoods, 2022)

Das elektrische Verhalten des Motors kann durch eine Ersatzschaltung beschrieben werden. Das wichtigste Merkmal eines Motors ist die Wicklung aus Kupferdraht, welches das Verhalten einer Spule aufweist und somit auch mittels einer Spule beschrieben werden kann. Außerdem hat der reale Elektromotor Verluste, die durch Kontaktübergangswiderstände und den Widerstand der Wicklung erzeugt werden, welches durch einen ohmschen Widerstand beschrieben werden kann. (heise, 2016)

Wenn der Motor sich dreht, ist er wie ein Generator und erzeugt eine Spannung U_M. Die erzeugte Spannung ist abhängig von der Drehzahl, dem Feld im Stator und der sogenannten Motorkonstante. (heise, 2016)

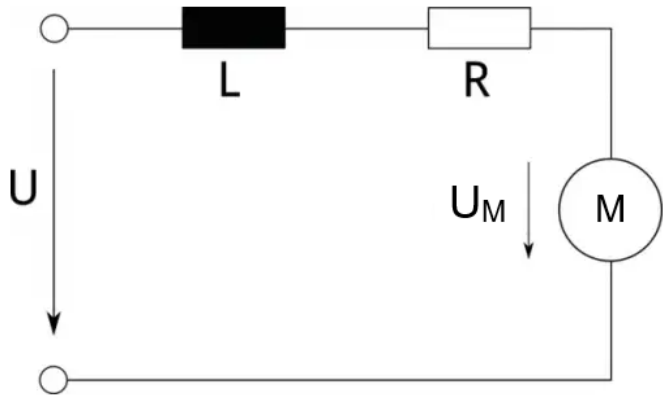


Abbildung 11: Ersatzschaltbild eines Schrittmotors (heise, 2016)

Mit folgender Messchaltung kann die Stromkennlinie über der Drehzahl erstellt werden.

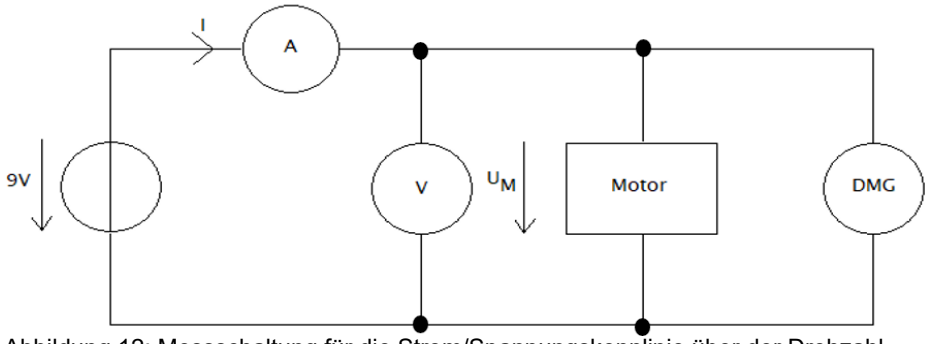


Abbildung 12: Messschaltung für die Strom/Spannungskennlinie über der Drehzahl

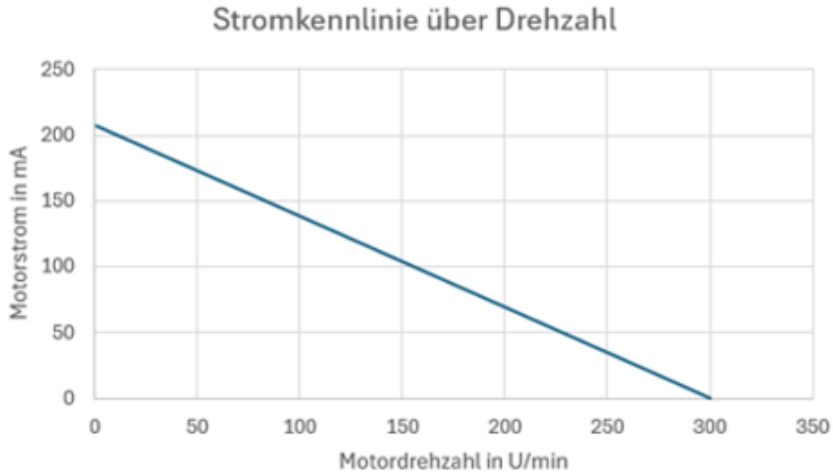


Abbildung 13: Stromkennlinie über Drehzahl

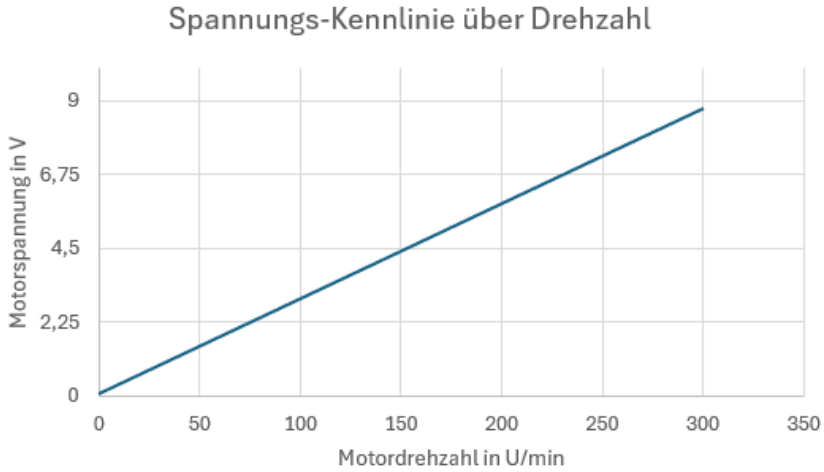


Abbildung 14: Spannungskennlinie über Drehzahl

Zu erkennen ist, dass die Motorspannung U_M proportional zur Drehzahl ist und somit im Stillstand 0 Volt beträgt. Bei maximaler Drehzahl ist U_M nahezu so groß ist wie die angelegte Spannung U. (heise, 2016)

Der maximale Strom fließt, wenn der Motor bei angelegter Versorgungsspannung noch nicht rotiert. Da die generierte Spannung $U_M = 0$ ist, wird der Strom nur durch den Verlustwiderstand R begrenzt. Da die Beschleunigung des Motors schnell ist, wird diese Phase beim Anlauf schnell überwunden. (heise, 2016)

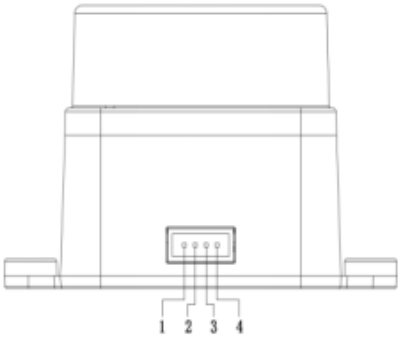
Umgebungserkennung: LD-19 360°- Entfernungssensor



Abbildung 15: Lidar-Sensor
(aliexpress, 2017)

Um die Umgebung des Fahrzeugs zu scannen, wird ein 360°-Sensor, in dem Fall der LD-19 verwendet.

Bestandteile des Sensors: Laser-Entfernungsmesskern, drahtlose Energieübertragungseinheit, drahtlose Kommunikationseinheit, Winkelmesseinheit, Motorantriebseinheit. (youyeetoo, 2017)



port number	signal name	type	description	minimum	typical	maximum
1	Tx	output	LiDAR data output	0V	3.3V	3.5V
2	PWM	input	motor control	0V	-	3.3V
3	GND	power supply	negative	-	0V	-
4	P5V	power supply	positive	4.5V	5V	5.5V

Tabelle 6: Sensor-Pinbelegung (youyeetoo, 2017)

4 Durchführung

4.1 Bluetooth-Kommunikation [David Kedra]

4.1.1 Ablauf

Ursprung

Ursprünglich war geplant, zwei Arduinos mit jeweils einem Bluetooth-Modul (HC-05) zu verwenden.

Das Programm dafür wurde oberflächlich funktionsfähig programmiert und getestet und der Code wurde in den folgenden Änderungen als Basis verwendet.

Testaufbau Bluetooth (Arduino Nano):

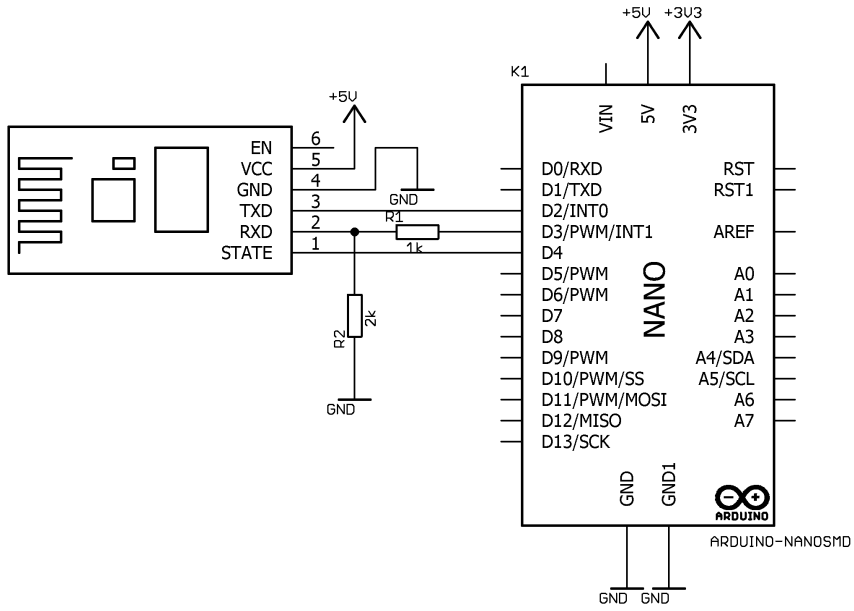


Abbildung 16: Testaufbau Bluetooth

Das Gerät links ist ein HC-05. Mit diesem Aufbau wurden softwaremäßig Testdaten von beiden Mikrocontrollern erfolgreich gesendet und empfangen.

Der Logikeingang `RXD` (Receive Data) und Logikausgang `TXD` (Transmit Data) des Bluetooth-Moduls HC-05 operieren mit 3,3 Volt. Daher wurde der Logikeingang des Moduls (`RXD`) mithilfe eines Spannungsteilers an den Arduino angeschlossen. Der Arduino liefert 5 Volt, die mit den Widerstandswerten $\mathrm{R1} = 1\mathrm{k}\Omega$ und $\mathrm{R2} = 2\mathrm{k}\Omega$ auf ungefähr 3,3 Volt heruntergeteilt werden.

$$U_a = 5V * \frac{R2}{R2 + R1} = 5V * \frac{2}{3} = 3,\dot{3}V \approx 3,3V$$

Es war vorübergehend in Ordnung, den Logikausgang des Moduls (`TXD`) mit dem Arduino Nano mit 3,3 Volt zu betreiben, da der Arduino diese Spannung gut genug lesen kann.

Es ist jedoch bekannt, dass diese Methode nicht optimal ist. Daher war es nicht geplant, diesen Aufbau in der endgültigen Version der Fernsteuerung beizubehalten und wurde nur während der Programmierung zum Testen verwendet.

Änderung 1: Einsatz von Raspberry Pi 4 und ESP32

Aufgrund der Erkenntnis, dass ein Arduino für unsere Anwendung in Bezug auf Leistung und Speicher unzureichend ist, haben wir uns entschieden, auf einen Raspberry Pi 4 für die Fernsteuerung und einen ESP32 für das Fahrzeug umzusteigen. Der Raspberry Pi 4 wurde gewählt, da kontinuierliches Senden und Empfangen von Daten per Bluetooth benötigt werden, gleichzeitig ein Bildschirm betrieben werden soll, eine große Menge an Umgebungsdaten gespeichert werden müssen und die komplexe Verarbeitung der Lidar-Daten durchgeführt werden muss. Der ESP32 wurde für das Fahrzeug gewählt, da er Daten per Bluetooth senden und empfangen soll, kontinuierlich Lidar-Daten empfangen und vier Motoren steuern muss.

Es wurde entschieden auf dem Raspberry Pi 4 mit C++ und nicht mit Python zu programmieren, sodass die Programmiersprache ident bei beiden Projektteilen bleibt, da das Bluetooth-Programm für beide Mikroprozessoren möglichst ident geschrieben werden soll.

a. Ohne Bluetooth-Modul

Trotz intensiver Versuche war es nicht möglich, die Kopplung über Bluetooth herzustellen. Aber nachdem es auf unerklärliche Weise gelungen war, war es nicht möglich, Daten manuell mittels C++-Code in beide Richtungen zu senden oder zu empfangen.

b. Mit Bluetooth-Modul

Die Verbindung mittels des zuvor verwendeten HC-05-Moduls über die UART-Schnittstelle funktionierte zuverlässig, ebenso das manuelle Senden und Empfangen zwischen dem Raspberry Pi 4 und dem ESP32.

Allerdings gab es ein Problem: Da die Fernbedienung einen TFT-Bildschirm ansteuern soll, muss dieser mit dem Raspberry Pi 4 kompatibel sein, welcher das „Raspberry Pi OS“, ein Linux basiertes Betriebssystem verwendet. Jedoch war es nicht möglich dafür eine Linux-kompatible Bibliothek für den spezifischen Treiber ILI9488 zu finden. Trotz ausgiebiger Suche und Versuche mit anderen Bibliotheken, die für ähnliche Bildschirme zur Verfügung gestellt wurden, wurde keine Lösung gefunden. Da die Entwicklung einer eigenen API für das Display sehr zeitaufwändig gewesen wäre, entschieden wir uns, wie im Fahrzeug, einen ESP32 zu verwenden.

Änderung 2: Einsatz von ESP32 für die Fernsteuerung

Der Wechsel zu einem ESP32 für die Fernsteuerung löste die Probleme, die wir mit einem Arduino hatten. Obwohl der ESP32 nicht so leistungsstark ist wie der Raspberry Pi 4, ist er für unser Projekt ausreichend. Die Umprogrammierung des Codes war unkompliziert, da die Arduino-IDE auch für den ESP32 verwendet werden kann und die Programmiersprache C++ dieselbe bleibt.

Nach der Umstellung funktionierte alles erfolgreich, ähnlich wie am Anfang mit zwei Arduinos.

Änderung 3 (Finale Version): Verzicht auf Bluetooth-Module

Es ist anzumerken, dass beide ESP32 über integrierte Bluetooth-Module verfügen, wodurch die Kommunikation ohne zusätzliche Bluetooth-Module möglich ist. Durch eine spezielle Umprogrammierung des Programms funktioniert die Verbindung nun einwandfrei ohne separate Bluetooth-Module. Diese Änderung führte zu einer Erhöhung der Übertragungsgeschwindigkeit, einer Verkürzung der Zeit bis zur Herstellung einer Verbindung und einer Verringerung der Sende- bzw. Empfangsverzögerung.

4.1.2 Datenformatierung und -übertragung

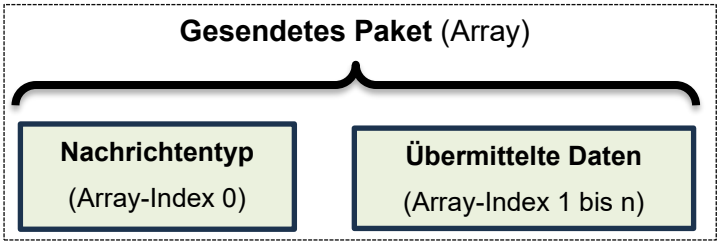


Abbildung 17: Bluetooth Datenformatierung

n … Anzahl der Werte in einem Paket

Gesendet wird ein Paket mit Daten. Die erste Information im Paket bestimmt den Nachrichtentyp, also um was es sich in den nachfolgenden Daten im Paket handelt. Jeder Nachrichtentyp hat eine vordefinierte Nummer, sodass der Empfänger ihn korrekt und eindeutig identifizieren kann. Die verbleibenden Teile des Pakets enthalten dann die tatsächlich übermittelten Daten.

Identifikation der Nachrichtentypen:

`0` → Joystick

`1` → Lidar

Da nur ASCII Zeichen einzeln versendet werden, gibt es spezielle Befehlszeichen, um zu wissen, wie die einzelnen Zahlen interpretiert werden sollen:

`\n` → Nächster Wert einer Nachricht

`\0` → Ende einer Nachricht

Realisierung im Code:

```
enum Message {
  // Befehlszeichen:
  END = '\0',   // Ende einer Nachricht
  NEXT = '\n',  // Nächster Wert einer Nachricht
  // Nachrichtentypen:
  JOYSTICK = 0,
  LIDAR = 1,
};
```

Bei einem unvollständigen oder fehlerhaften Paket wird je nach Art des Fehlers eine Fehlermeldung ausgelöst und das Paket wird übersprungen.

Programmierte Fehlererkennungen sind:

- ein unbekannter Nachrichtentyp
- die Länge der eingehenden Nachricht ist größer als die vordefinierte maximale Nachrichtenlänge
- das empfangene Paket entspricht nicht der Paketform des Nachrichtentyps
- das empfangene ASCII Zeichen ist weder eine Ziffer noch ein vordefiniertes Befehlszeichen.

Beispiele für die Übermittlung von Werten

Joystick

Was gesendet wird: `0, \n, 5, 7, \n, 1, 4, 3, \0`

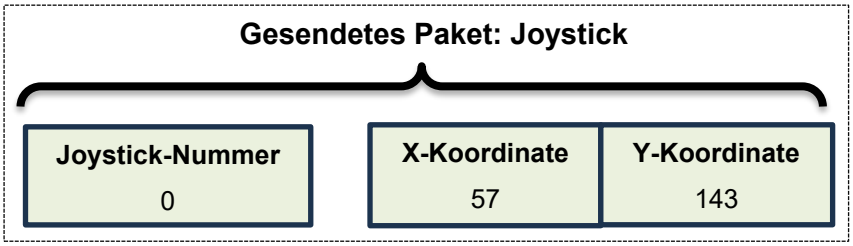


Abbildung 18: Bluetooth Datenformatierung Joystick

Lidar

Was gesendet wird: `1, \n, 1, 3, 2, \n, 1, 8, 1, \0`

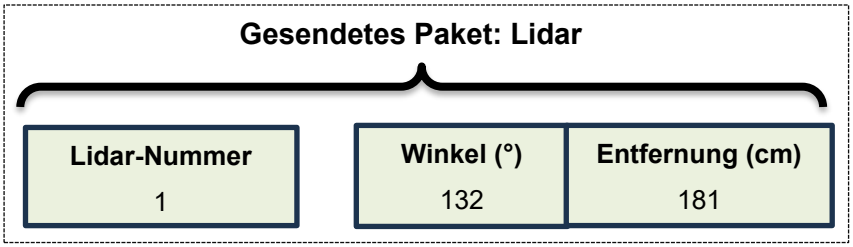


Abbildung 19: Bluetooth Datenformatierung Lidar

4.1.3 Bluetooth-Kopplung

Bei einer Bluetooth-Verbindung gibt es immer einen Server und einen Client. Der Server kontrolliert die Kommunikation und übernimmt dabei die Koordination. In diesem Projekt wurde festgelegt, dass bei der Bluetooth-Kommunikation die Fernsteuerung als Server und das Fahrzeug als Client agiert.

Im Verbindungsaufbau versuchen sich die beiden Bluetooth-Geräte anhand ihrer Gerätenamen zu identifizieren, um sich mit dem richtigen Gerät zu koppeln. Der Fernsteuerung wurde der Name `ESP32-BT-Controller` zugewiesen und dem Fahrzeug der Name `ESP32-BT-Car`.

```
String masterName = "ESP32-BT-Controller";
String slaveName = "ESP32-BT-Car";
```

Der Server verbindet sich mit dem Client, indem er eine Kopplungsanfrage sendet, auf die der Client wartet. Für beide Geräte muss eine Rückruf-Funktion erstellt und registriert werden mittels der `register_callback` Funktion.

Kopplungsablauf Server:

```
SerialBT.register_callback(Bt_Status); // Rückruf-Funktion registrieren
SerialBT.begin(masterName, true); // Bluetooth starten als Master
SerialBT.connect(slaveName); // Versuchen mit Slave zu verbinden
```

Kopplungsablauf Client:

```
SerialBT.register_callback(Bt_Status); // Rückruf-Funktion registrieren
SerialBT.begin(slaveName, false); // Bluetooth starten als Slave
```

Die für den Rückruf registrierte Funktion `Bt_Status` wird bei jedem auftretenden Bluetooth-Ereignis ausgeführt. Dabei sind drei Bluetooth-Ereignisse von Interesse:

- Verbindung hergestellt als Server (`ESP_SPP_OPEN_EVT`)
- Verbindung hergestellt als Client (`ESP_SPP_SRV_OPEN_EVT`)
- Verbindung getrennt (`ESP_SPP_CLOSE_EVT`)

Die Form der Rückruf-Funktion ist vordefiniert und ident für beide Rollen:

```
void Bt_Status(esp_spp_cb_event_t event, esp_spp_cb_param_t* param);
```

In diesem Fall ist nur der Parameter `event` relevant, da man damit feststellen kann, welchen Ereignis aufgetreten ist.

Der Inhalt der Funktion variiert je nach Rolle.

Inhalt der Funktion `Bt_Status` beim Server (Fernsteuerung):

```
if (event == ESP_SPP_OPEN_EVT) {
  // Verbindung hergestellt (als Master)
  isConnected = true;
} else if (event == ESP_SPP_CLOSE_EVT) {
  // Verbindung getrennt
  isConnected = false;
}
```

Inhalt der Funktion `Bt_Status` beim Client (Fahrzeug):

```
if (event == ESP_SPP_SRV_OPEN_EVT) {
  // Verbindung hergestellt (als Slave)
  isConnected = true;
} else if (event == ESP_SPP_CLOSE_EVT) {
  // Verbindung getrennt
  isConnected = false;
}
```

Der aktuelle Verbindungsstatus wird in der Variable `isConnected` gespeichert, um jederzeit den Status abrufen zu können.

Es kann vorkommen, dass die Verbindung mit dem Client nicht beim ersten Mal erfolgreich ist. Das kann mehrere Gründe haben, zum Beispiel wenn das Gerät noch nicht in Betrieb ist. Deshalb wurde bei der Fernsteuerung (Server) zusätzlich programmiert, dass jede Sekunde versucht wird, sich mit dem Fahrzeug (Client) zu verbinden. Diese Logik wurde in der `IsConnected` Funktion geschrieben, die eigentlich dazu dient den Wert der Variable `isConnected` zu liefern.

Um die Verwendung von einem „delay“ zu vermeiden, wird ein Timer verwendet. Hierbei kommt die Funktion `millis()` zum Einsatz, welche die Anzahl der Millisekunden liefert, die seit dem Start des Programms vergangen sind.

```
const int PING_INTERVAL = 1000; // in Millisekunden
unsigned long pingTimerMax = 0; // Timer-Variable

bool IsConnected() {
  if (!isConnected && millis() > pingTimerMax) {
    // Timer-Variable aktualisieren (aktuelle Zeit + Intervallzeit):
    pingTimerMax = millis() + PING_INTERVAL;
    SerialBT.end(); // Bluetooth deaktivieren
    SerialBT.begin(masterName, true); //Bluetooth starten als Master
    SerialBT.connect(slaveName); // Versuchen mit Slave zu verbinden
  }
  return isConnected; // aktuellen Verbindungsstatus zurückgeben
}
```

Diese Funktion `IsConnected` wird in der loop-Schleife des Mikrocontrollers ausgeführt. Der darauf nachfolgende Code wird nur dann ausgeführt, wenn eine Bluetooth-Verbindung besteht.

```
void loop() {
  if (!bt.IsConnected()) return;
  // Code, der nur bei einer Bluetooth-Verbindung ausgeführt wird.
}
```

4.1.4 Algorithmus für das Empfangen

In diesem Abschnitt wird die programmierte Funktion `HandleIncomingMessage` erklärt. Diese Funktion ist identisch für beide Projektteile, die Fernsteuerung und das Fahrzeug.

Funktionsbeschreibung

```
bool HandleIncomingMessage(int* outMessage, int sizeofOutMessage);
```

Die Funktion liefert einen Boolean-Wert zurück. Sie gibt nur dann `true` zurück, wenn eine vollständige und korrekte Nachricht empfangen wurde, andernfalls `false`. Die empfangene Nachricht wird dann in `outMessage` gespeichert. `outMessage` ist einer der beiden Parameter der Funktion mit dem Datentyp `int*`. Er dient dazu, ein Integer-Array zu übergeben, worin die empfangene Nachricht gespeichert wird. Deswegen wird es als ein Pointer übergeben. Der zweite Parameter ist `sizeofOutMessage` mit dem Datentyp `int` und repräsentiert die Größe des übergebenen Integer-Arrays `outMessage` in Bytes. Dieser wird wie folgt übergeben: `sizeof(outMessage)`.

Überprüfung vor dem Empfangen

Bevor das eigentliche Empfangen und die Formatierung stattfindet wird überprüft, ob aktuell eine Bluetooth-Verbindung besteht und ob die Größe von `outMessage` mit der eingestellten Bluetooth-Nachrichtengröße übereinstimmt. Wenn nicht, wird eine spezielle Fehler-Nachricht in der Konsole ausgegeben und es wird `false` zurückgegeben. Durch die Verwendung von `return` wird der Rest der Funktion nicht ausgeführt.

```
if (!isConnected) {
  return false;
}
if (sizeofOutMessage != sizeof(incomingMessage)) {
  return false;
}
// Rest der Funktion
```

Nach der Überprüfung der Bluetooth-Verbindung und der Größe der eingehenden Nachricht wird im nächsten Schritt die Nachricht empfangen

Empfang der Nachricht

Das im ESP32 integrierte Bluetooth-Modul kommuniziert per serieller Schnittstelle mit dem Mikroprozessor. Die Logik für das Empfangen ist mit der integrierten Bibliothek in der Arduino IDE leicht umsetzbar. Mit `SerialBT.available()` erhält man die Anzahl der verfügbaren Daten auf der seriellen Schnittstelle. Mit `SerialBT.read()` wird das erste Byte gelesen und automatisch aus der Warteschlange der seriellen Schnittstelle entfernt. Dieses wird in den Datentyp `char` umgewandelt, da mit jedem Byte ein 8-Bit-ASCII-Zeichen versendet wird.

```
while (SerialBT.available() > 0) {
    char receivedValue = SerialBT.read();
    // Code für die Formatierung
}
return false;
```

Nach der while-Schleife wird `false` zurückgegeben, was bedeutet, dass keine Nachricht empfangen wurde.

Formatierung

Die Formatierung besteht grundlegend aus drei if-Verzweigungen.

1. **Der empfangene Wert entspricht dem Befehlszeichen `NEXT`[1]**

   Das bedeutet, dass die folgenden Daten den nächsten Wert einer Nachricht darstellen.

   Innerhalb der if-Verzweigung wird die Variable `messagePos` inkrementiert. Diese Variable definiert die aktuelle Position eines Pakets, also welcher Wert gerade gelesen wird.

   ```
   if (receivedValue == Message::NEXT) {
     // nächster Wert der Nachricht
     messagePos++;
   }
   ```

[1] Für weitere Informationen zu den Befehlszeichen siehe Kapitel „Datenformatierung und -übertragung“ auf Seite 23.

2. **Der empfangene Wert entspricht dem Befehlszeichen `END`**

   Das bedeutet, dass die Nachricht vollständig empfangen wurde.

   Innerhalb der if-Verzweigung wird nun überprüft, ob die Anzahl der Werte der eingehenden Nachricht entsprechend korrekt ist. Ist das nicht der Fall wird eine Fehlermeldung ausgegeben und es wird `false` zurückgegeben. Die Fehlernachricht lautet wie folgt: „*ERROR: Anzahl der Werte in eingehender Nachricht ist länger als die eingestellte MAX_MESSAGE_LENGTH! Die Nachricht wird übersprungen.*“.

   Andernfalls wurde die korrekte und vollständige Nachricht empfangen. Diese wird nun im Parameter `outMessage` der Funktion gespeichert und es wird `true` zurückgegeben.

```
else if (receivedValue == Message::END) {
  // vollständige Nachricht erhalten
  if (messagePos >= MAX_MESSAGE_LENGTH) {
    // Anzahl der Werte in empfangener Nachricht ist zu groß
    ClearIncomingMessage();
    return false;
  } else {
    // korrekte vollständige Nachricht erhalten, Nachricht wird gespeichert
    memcpy(outMessage, incomingMessage, sizeof(incomingMessage));
    ClearIncomingMessage();
    return true;
  }
}
```

3. **Eine Ziffer wurde empfangen**

   Zum Überprüfen, ob das empfangene Zeichen eine Ziffer ist, wird die in C++ enthaltene Funktion `isdigit` angewendet.

   Innerhalb der if-Verzweigung muss das empfangene 8-Bit-ASCII-Zeichen in eine Ziffer umgewandelt werden. Das geschieht durch die Subtraktion des ASCII-Werts des Zeichens „0“ vom ASCII-Wert des empfangenen Zeichens: `receivedValue - '0'`. Der ASCII-Wert von „0“ ist 48. Wenn beispielsweise das empfangene Zeichen

„3“ ist, was einen ASCII-Wert von 51 hat, ergibt die Berechnung $51 - 48 = 3$. Dies wandelt das ASCII-Zeichen „3“ in den Integer-Wert 3 um.

Dann muss die neue Ziffer an das Ende der aktuellen Zahl in der Nachricht angehängt werden. Das geschieht, indem die aktuelle Zahl in der Nachricht um eine Stelle nach links verschoben wird, durch die Multiplikation mit 10, und die neue Ziffer `digit` dazu addiert wird.

```
else if (messagePos < MAX_MESSAGE_LENGTH && isdigit(receivedValue)) {
  // Datentyp `char` zu `int` konvertieren
  int digit = receivedValue - '0';
  // Ziffer an das Ende der aktuellen Zahl in der Nachricht anhängen
  incomingMessage[messagePos] = incomingMessage[messagePos] * 10 + digit;
}
```

4.1.5 Algorithmus für das Senden

In diesem Abschnitt wird die programmierte Funktion `SendMessage` erklärt. Diese Funktion ist identisch für beide Projektteile, die Fernsteuerung und das Fahrzeug.

Zum Senden wird die Nachricht in einzelne Ziffern aufgeteilt und so gespeichert.

```
int digitPos = 0;
int tmpMessage = message[i];

while (tmpMessage > 0) {
  int digit = tmpMessage % 10; // rechte Ziffer entnehmen
  digits[digitPos++] = digit; // rechte Ziffer speichern
  tmpMessage /= 10; // Die Position der Ziffern nach rechts verschieben
}
```

Durch das Modulo-Rechnen durch zehn kann die ganz rechte Ziffer entnommen werden, da diese immer der Rest nach der Division durch zehn ist.

Durch die Division der Ganzzahl `tmpMessage` durch zehn wird die Position der Ziffern nach rechts verschoben, sodass die zuvor ganz rechte Ziffer entfernt wird, da keine Dezimalstellen gespeichert werden.

Als nächstes werden die Ziffern in die richtige Reihenfolge gebracht, indem das Array umgekehrt wird. Die Ziffern werden nacheinander zu einem ASCII-Zeichen durch Addition von `'0'` umgewandelt[2] und anschließend per Bluetooth versendet.

```
for (int j = digitPos - 1; j >= 0; j--) { // Umgekehrte for-Schleife
  char digitAsAscii = '0' + digits[j]; // Ziffer in ASCII Zeichen umwandeln
  SerialBT.write(digitAsAscii); // Ziffer einzeln senden
}
```

Nach Abschluss jedes Nachrichtenteils wird diese Information mit der Anweisung `SerialBT.write(Message::NEXT)` versendet.

Die Darstellung der Information für das Ende der gesamten Nachricht erfolgt mit der Anweisung `SerialBT.write(Message::END)`.

[2] Warum eine Ganzzahl durch Addition von `'0'` in ein ASCII-Zeichen umgewandelt wird, siehe Kapitel „Algorithmus für das Empfangen“ unter „Formatierung“ und „3. Eine Ziffer wurde empfangen“.

4.2 Umgebungsberechnung und -speicherung [David Kedra]

4.2.1 Einleitung

Die Speicherung erfolgt nur, wenn das Fahrzeug stationär ist. Wenn das Fahrzeug fährt, wird die Speicherung zurückgesetzt.

Der Grund dafür ist, dass das Fahrzeug seine absolute Position im Raum nicht kennt und keine genauen Werte bei einer Bewegung oder Drehung gemessen werden. Dies erschwert die exakte Bestimmung der tatsächlichen Bewegung im Raum. Dazu wäre ein angepasster Algorithmus erforderlich, der genau für diesen Zweck entwickelt wurde. Als potenzielle Lösung wurde der sogenannte SLAM-Algorithmus in Betracht gezogen. *„Als SLAM (englisch Simultaneous Localization and Mapping; deutsch Simultane Positionsbestimmung und Kartierung) wird ein Verfahren der Robotik bezeichnet, bei dem ein mobiler Roboter gleichzeitig eine Karte seiner Umgebung erstellen und seine räumliche Lage innerhalb dieser Karte schätzen muss.“* (Wikipedia, 2024).

Die Idee der Einbindung des Algorithmus führte jedoch zu einigen Schwierigkeiten:

- Der ESP32 Mikroprozessor hat eine begrenzte Leistungsfähigkeit, welche nach der Implementierung des rechenintensiven Algorithmus SLAM zu Problemen führen würde.
- Die Suche nach einem geeigneten, bereits vorhandenen Algorithmus passend für die spezifischen Anforderungen des ESP32 ist herausfordernd.
- Die Integration des komplexen Algorithmus SLAM in den bestehenden Code würde den Entwicklungsaufwand stark erhöhen.

Auch nach Entdeckung des Algorithmus tinySLAM[3] welcher in unter 200 Zeilen Code programmiert wurde, wurde es trotzdem wegen dem genannten Punkt Drei nicht genutzt, denn es benötigt zusätzliche Odometrie, also eine „Schätzung von Position und Orientierung (Lageschätzung) eines mobilen Systems anhand der Daten seines Vortriebsystems.“ (Wikipedia, 2024). Dies zu integrieren, würde sehr viel Zeit in Anspruch nehmen und nicht die optimale Funktionalität des Systems garantieren.

[3] https://openslam-org.github.io/tinyslam.html

Aus diesen Gründen und Problemen wurde nach Information über den Algorithmus und einiger Suche nach passendem Code die Funktion der Umgebungsspeicherung des Projekts geändert, sodass die Speicherung nur im Stillstand erfolgt.

4.2.2 Klassenbeschreibung

Die Klasse `LidarMapper` ist verantwortlich für die Berechnung von Objektpositionen und damit der Umgebung in der Form einer zweidimensionalen Karte und zur Aktualisierung der Karte basierend auf Lidar-Daten.

Es wird eine Struktur für die Position verwendet, um die x-Koordinate und y-Koordinate einer Position zusammenhängend speichern zu können.

```
struct Position {
  int x, y;
};
```

Zur Repräsentierung der Zustände an einer Position wird ein Enumerator verwendet.

- `AIR`: An dieser Position ist eine leere Stelle bzw. Luft.
- `OBJECT`: An dieser Position ist ein erkanntes Objekt.
- `UNKNOWN`: Der Zustand dieser Position ist nicht bekannt, z.B. hinter einem Objekt.

```
enum PositionState {
  AIR, OBJECT, UNKNOWN
};
```

Die Speicherung der Karte erfolgt auf der Grundlage eines zweidimensionalen Vektors, der wie ein 2D-Array funktioniert. Der Datentyp, der zur Speicherung verwendet wird, ist `PositionState`. Die Deklaration sieht wie folgt aus.

```
vector<vector<PositionState>> map;
```

Im Konstruktor wird die Kartenansicht initialisiert mit der Größe `MAP_SIZE_X` zu `MAP_SIZE_Y` und das gesamte Array wird auf den Wert `UNKNOWN` gesetzt.

```
map = vector<vector<PositionState>>(MAP_SIZE_X, vector<PositionState>(MAP_SIZE_Y, UNKNOWN));
```

4.2.3 Datenverarbeitung

Die Funktion `calcPosition(int angle, int distance)` berechnet die Position eines Objekts basierend auf dem Winkel und der Entfernung zum Lidar-Sensor. Es werden trigonometrische Funktionen zur Umrechnung von Polarkoordinaten in kartesische Koordinaten verwendet. Anschließend wird die Position auf den Bereich der Kartenansicht skaliert und der Nullpunkt zentriert. Für mehr Information zum Ablauf dieser Funktion siehe Kapitel „Optimierung der Umgebungsspeicherung“ auf Seite 41.

Die Funktion `calcObjectAndUpdateMap(int angle, int distance)` berechnet die Position eines erkannten Objekts und aktualisiert die Kartenansicht entsprechend.

Es wird der Zustand vom erkannten Objekt auf `OBJECT` gesetzt und der Bereich entlang einer Linie vom erkannten Objekt bis zum Lidar-Sensor wird auf den Wert `AIR` gesetzt. Dies erfolgt, indem die Entfernung des erkannten Objekts schrittweise minimiert wird, wobei der Winkel gleichbleibt, und die `calcPosition` genutzt wird, um nacheinander die Koordinaten der Luft zwischen Objekt und Lidar-Sensor berechnet werden und auf AIR gesetzt werden. Ein Schritt entspricht der minimalen Änderung der Distanz, sodass eine Änderung in der Speicherung möglich ist. Dies wird wie folgt berechnet.

```
// Berechnung der minimalen Änderung der Distanz
int stepSize_x = 2 * MAX_DISTANCE / MAP_SIZE_X;
int stepSize_y = 2 * MAX_DISTANCE / MAP_SIZE_Y;
stepSize = min(stepSize_x, stepSize_y);
```

Die Berechnung und Setzen der Luft zwischen Sensor und Objekt verläuft wie folgt.

```
// Berechnung und Setzen der Luft zwischen Sensor und Objekt
while (distance > 0) {
  pos = calcPosition(angle, distance); // Berechnung von x und y Position
  if (pos.x >= 0 && pos.x < MAP_SIZE_X && pos.y >= 0 && pos.y < MAP_SIZE_Y) {
    // innerhalb der Arraygrenzen
    if (map[pos.x][pos.y] == UNKNOWN || map[pos.x][pos.y] == OBJECT) {
      // Position ist nicht Luft
      map[pos.x][pos.y] = AIR; // Position auf Luft setzen
    }
  }
  distance -= stepSize; // nächster Schritt
}
```

4.2.4 Optimierung der Umgebungsspeicherung

Dieser Teil befasst sich mit der Funktion `calcPosition` innerhalb der Klasse `LidarMapper`.

Problem

Nach Tests durch Messung der Zeit wurde festgestellt, dass die Funktion möglicherweise nicht gut optimiert ist, da die Berechnung ziemlich lange dauert. Das kann zu Problemen führen, da der Mikrocontroller auch die Bildschirmanzeige durchführt, die viel Zeit in Anspruch nimmt und gleichzeitig Joystick-Daten per Bluetooth an das Fahrzeug sendet, womit eine flüssige Steuerung möglich sein soll.

Tests haben gezeigt, dass eine bestimmte Berechnung viel Zeit in Anspruch nimmt, und zwar den größten Teil der gesamten Ausführung der Funktion.

```
// Konvertieren von polar- in kartesische Koordinaten
float xFloat = distance * cos(angle * M_PI / 180);
float yFloat = distance * sin(angle * M_PI / 180);
// Koordinaten auf Array Größe anpassen
pos.x = map(xFloat, 0, MAX_DISTANCE, 0, MAX_POSITION_X / 2);
pos.y = map(yFloat, 0, MAX_DISTANCE, 0, MAX_POSITION_Y / 2);
// Mittelpunkt in die Mitte des Arrays setzen
pos.x += MAX_POSITION_X / 2;
pos.y += MAX_POSITION_Y / 2;
```

Die Funktion `map` konvertiert eine Zahl „`x`“ aus dem Zahlenbereich „`in`“ in den Zahlenbereich von „`out`“ und gibt diese zurück. Die Zahlenbereiche werden mit jeweils einem minimalen und einem maximalen Wert definiert. Diese Funktion ist in der `Arduino.h` Bibliothek enthalten, welche ohne ein zusätzliches `include` in einer „`.ino`“-Datei automatisch hinzugefügt wird.

```
int map(int x, int in_min, int in_max, int out_min, int out_max);
```
[4]

[4] https://www.arduino.cc/reference/en/language/functions/math/map/

Zurück zu meiner Funktion, wie aus den Kommentaren ersichtlich, wandelt der erste Teil die Koordinaten von Polarform in kartesische Form um. Der zweite Teil konvertiert die Koordinaten `xFloat` und `yFloat` vom Zentimeterbereich (`0` bis `MAX_DISTANCE`) in den Bereich einer definierten Speicherauflösung (`0` bis `MAX_POSITION_X / 2`). Im dritten Teil wird die Hälfte der maximalen Speicherauflösung dazu addiert, um den Koordinatenursprung in die Mitte zu setzen.

Durch Tests konnte ich die Ausführungszeit der Funktion `LidarMapper::calcObjectAndUpdateMap` ermitteln, in der sich dieser Code befindet.

Testfunktion:

```
unsigned long start = millis();
distance = 80000;
for (float angle = 0; angle < 360; angle += 0.5) {
  mapper.calcObjectAndUpdateMap(distance, angle);
}
Serial.println(millis() - start);
```

Hinweis: • `mapper` ist ein Objekt der Klasse `LidarMapper`.

• `millis()` liefert die Anzahl der Millisekunden, die seit dem Start des Programms vergangen sind.

Bei der Ausführung dieser Testfunktion auf dem ESP32 wurden ca. **2200 ms** gemessen.

1. Durch Weglassen der Konvertierung von polar- in kartesische Koordinate, also das Entfernen der Kosinus- und Sinusberechnung reduzierte sich die Zeit auf ca. **2000 ms**. Das ergibt eine maximal mögliche Zeitersparnis von **9,09%.**

2. Nach Weglassen der Berechnung für das Anpassen der Koordinaten auf die Array Größe und setzen der Mitte des Arrays in den Mittelpunkt reduziert sich diese Zeit auf von **2000 ms** auf ca. **170 ms**. Das ergibt eine maximal mögliche Zeitersparnis von **91,5%.** Es ist also klar, dass die meiste Zeit der gesamten Funktion in dieser Berechnung liegt.

Problemlösung 1: Sinus und Kosinus

Da sich die Berechnung bei gleichen Winkeln häufig wiederholt und dabei nur die Distanz verringert wird, kann man den berechneten Kosinus und Sinus des Winkels speichern und nur dann neu berechnen, wenn sich der Winkel geändert hat.

Mein Lösungsansatz war es, statische Variablen zur Speicherung dieser beiden Werte sowie des vorherigen Winkels zu verwenden, um zu wissen, wenn dieser sich ändert.

Optimierter Code:

```
static float previousAngle = -1;
static float cosAngle = -1;
static float sinAngle = -1;

if (angle != previousAngle) {
  cosAngle = cos(angle * PI / 180);
  sinAngle = sin(angle * PI / 180);
  previousAngle = angle;
}
// von Polarform in kartesische Form
float xFloat = distance * cosAngle;
float yFloat = distance * sinAngle;
```

Durch diese Verbesserung wurde die Testzeit von **2200 ms** auf ungefähr **2030 ms** reduziert. Das ergibt eine Zeitersparnis von **7,73%**.

Problemlösung 2: pos.x und pos.y

In dieser Problemlösung wurde als erster Schritt die Berechnungen in jeweils nur eine Gleichung vereinfacht.

Dadurch ergibt sich Folgendes:

```
pos.x = 0.5 * (xFloat / MAX_DISTANCE + 1) * MAX_POSITION_X;
pos.y = 0.5 * (yFloat / MAX_DISTANCE + 1) * MAX_POSITION_Y;
```

Dieser Schritt führte bereits zu einer Reduzierung der Zeit der Testfunktion von **2030 ms** auf **1700 ms**. Das ergibt eine Zeitersparnis von **16,26%**.

In einem weiteren Schritt wurden die Berechnungen mit konstanten Werten getrennt, um sie nur einmal in der Initialisierung ausführen zu müssen und somit den Rechenaufwand zu verringern.

Nach der Trennung der Gleichungen: außerhalb der Funktion:

```
float halfMaxPosX = 0.5 * MAX_POSITION_X;
float halfMaxPosY = 0.5 * MAX_POSITION_Y;
float scaleFactorX = halfMaxPosX / MAX_DISTANCE;
float scaleFactorY = halfMaxPosY / MAX_DISTANCE;
```

Nach der Trennung der Gleichungen: innerhalb der Funktion:

```
pos.x = xFloat * halfMaxPosX + scaleFactorX;
pos.y = yFloat * halfMaxPosY + scaleFactorY;
```

Durch diesen weiteren Schritt konnte die Testzeit von **1700 ms** auf **1500 ms** reduziert werden. Das ergibt eine Zeitersparnis von **11,76%**.

Die insgesamte Zeitersparnis der Problemlösung 2 (von **2030 ms** auf **1500 ms**) beträgt **26,11%.**

4.3 Routenfindung [David Kedra]

Zur Berechnung wird der Lee-Algorithmus[5] angewendet. Dieser kalkuliert den kürzesten Weg zwischen zwei Punkten mit Berücksichtigung von Hindernissen.

Da im Rahmen des FSST-Unterrichts wir den Arbeitsauftrag zur Programmierung des Lee-Algorithmus hatten, wurde die Logik des Codes von mir bereits in C++ geschrieben. Dieser Code musste jedoch angepasst werden, um den Anforderungen des Projekts zu entsprechen. Diese Anpassung nimmt weniger Zeit in Anspruch als die vollständige Programmierung der Funktion von Grund auf.

Bildliche Darstellung des Algorithmus anhand eines Beispiels des vorhandenen Codes:

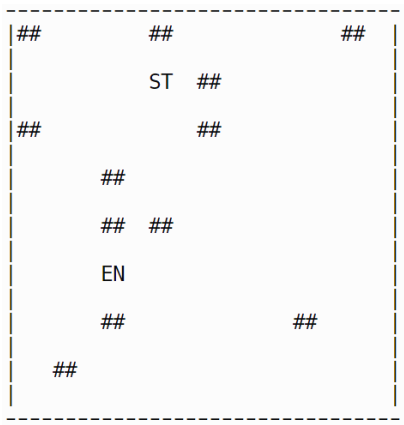


Abbildung 20: Lee-Alg., Ausgangslage

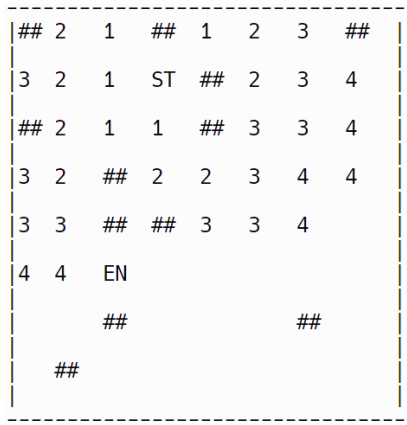


Abbildung 21: Lee-Alg., Kalkulation

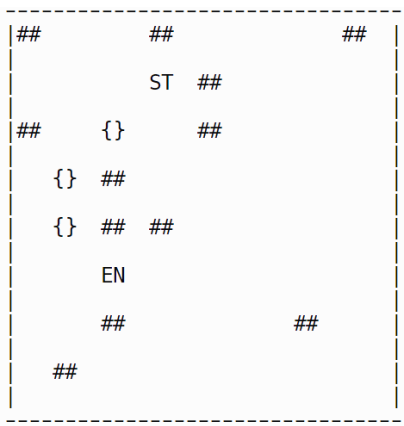


Abbildung 22: Lee-Alg., Ergebnis

ST … Start
EN … Ende / Ziel
… Objekt / Hindernis
{} … Route

Die Zahlen in der Abbildung der Kalkulation repräsentieren die Schritte der Ausbreitung vom Startpunkt aus. Jede Zahl gibt die Entfernung zum Startpunkt an. Der Algorithmus markiert jede von seiner Position aus erreichbarer Zelle, mit der Anzahl der Schritte, die benötigt werden, um sie vom Startpunkt aus zu erreichen, wobei Hindernisse nicht markiert werden. Sobald die Endposition erreicht ist, kehrt der Algorithmus zur Startposition zurück, wobei er die Zellen wählt, deren markierte Zahl geringer ist als die aktuelle Zahl. Anschließend zeigt das Ergebnis den Pfad mit der geringsten Anzahl von Schritten zum Ziel. In diesem Fall sind vier Schritte erforderlich, um das Ziel zu erreichen.

[5] https://de.wikipedia.org/wiki/Lee-Algorithmus

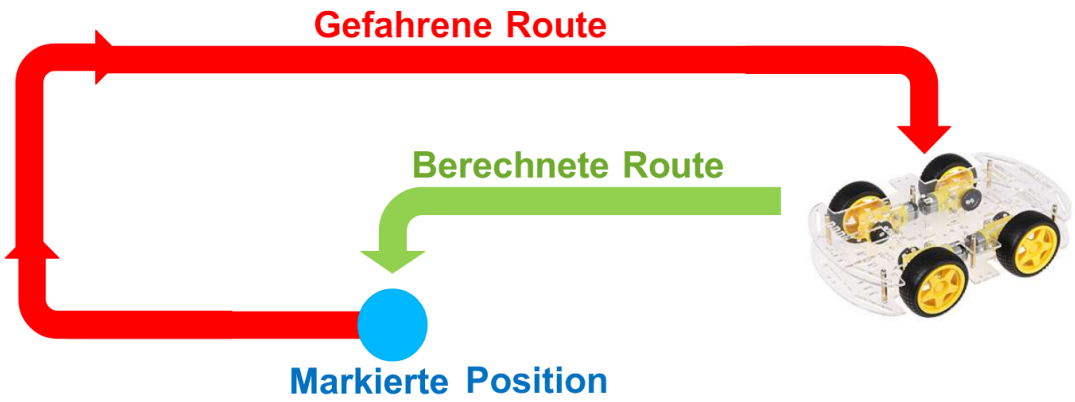


Abbildung 23, Berechnete Route zur markierten Position

```
int currentNumber = 0;
  while (true) {
    // alle Zellen der Karte durchlaufen
    for (int x = 0; x < ROW_SIZE; x++) {
      for (int y = 0; y < COLUMN_SIZE; y++) {
        if ((map[x][y] == currentNumber && currentNumber != 0)
            || (currentNumber == 0 && map[x][y] == START)) {
          // aktuelle Zelle gefunden
          // die Nachbarzellen der aktuellen Zelle durchlaufen (inklusive diagonal)
          for (int i = x - 1; i <= x + 1; i++) {
            for (int j = y - 1; j <= y + 1; j++) {
              if (i < 0 || j < 0 || i > ROW_SIZE - 1 || j > COLUMN_SIZE - 1) {
                // außerhalb des Arrays
                continue; // nächster Wert
              }
              if (map[i][j] == 0) {
                // Nachbarzelle noch nicht markiert
                map[i][j] = currentNumber + 1; // Nachbarzelle mit der nächsten Nummer markieren
              } else if (map[i][j] == END) {
                // Ziel erreicht
                return currentNumber + 1; // Rückgabe der Weglänge
              }
            }
          }
        }
      }
    }
    currentNumber++;
    if (currentNumber > ROW_SIZE * COLUMN_SIZE) {
      return -1; // Kein Weg wurde gefunden
    }
  }
```

Der Algorithmus durchläuft die zweidimensionale Karte, beginnend mit dem Startpunkt. Für jede Zelle wird eine Überprüfung der benachbarten Zellen durchgeführt und eine Markierung entsprechend vorgenommen.

Wenn eine benachbarte Zelle noch nicht markiert ist, wird sie mit der aktuellen Nummer plus eins markiert. Wenn eine benachbarte Zelle das Endziel ist, wird die Länge des gefundenen Weges zurückgegeben.

Wurden alle Nachbarszellen der aktuellen Zellenzahl markiert, wird die aktuelle Zahl der Überprüfungszelle inkrementiert und seine Nachbarszellen überprüft.

Der Algorithmus wird so lange wiederholt, bis entweder das Endziel erreicht oder alle erreichbaren Zellen markiert wurden. Falls das Endziel nicht erreicht wird und alle Zellen markiert sind, ohne das Ziel zu finden, wird die Zahl -1 zurückgegeben, um anzuzeigen, dass kein Weg möglich ist.

4.4 Darstellung auf dem Bildschirm [David Kedra]

Zur Darstellung wird die Bibliothek „ILI9486_SPI“[6] genutzt, welche den Treiber ILI9488 des Display-Moduls unterstützt. Zur Darstellung wird eine Adafruit-Library[7] genutzt, welche in der „ILI9486_SPI“ Bibliothek mitintegriert ist und eine Vielzahl an Funktionen bietet, die sich einfach verwenden lassen, um verschiedene Elemente auf dem Bildschirm zu zeichnen oder Text anzuzeigen.

Die ganze Funktion der Darstellung wurde innerhalb einer Klasse mit dem Namen `TFTDisplay` programmiert.

4.4.1 Initialisierung und Klassenbeschreibung

Farben werden mit einer 4-stelligen Hexadezimalzahl dargestellt. Zum Beispiel steht `0x0000` für Schwarz, `0x001F` für Blau und `0xF800` für Rot. Insgesamt wurden acht Farben definiert, für eine einfachere und verständlichere Programmierung.

```
#define BLACK 0x0000
#define BLUE 0x001F
#define RED 0xF800
...
```

Es werden zunächst Variablen definiert.

- `TFT_WIDTH` und `TFT_HEIGHT`, welche die Größe der Anzeige in Pixeln angeben. Diese beträgt 480 x 320.
- `CELL_WIDTH` und `CELL_HEIGHT` definieren die Dimensionen einer Zelle. Da die Größe der Speicherauflösung nicht identisch mit der Bildschirmauflösung ist, werden mehrere Pixel für einen Wert des Speichers verwendet. Diese werden als Zelle bezeichnet und wie im Codeausschnitt unten ersichtlich berechnet.
- Drei vordefinierte Farben für alle drei möglichen Zustände Objekt, Luft und Unbekannt. Zur Speicherung wird ein `uint16_t` verwendet, da eine Farbe ein positiver 16-Bit-Datentyp ist.

[6] https://github.com/ZinggJM/ILI9486_SPI
[7] https://cdn-learn.adafruit.com/downloads/pdf/adafruit-gfx-graphics-library.pdf

```
const int TFT_WIDTH = 480;
const int TFT_HEIGHT = 320;

const int CELL_WIDTH = TFT_WIDTH / MAP_SIZE_X;
const int CELL_HEIGHT = TFT_HEIGHT / MAP_SIZE_Y;

const uint16_t OBJECT_COLOR = BLACK;
const uint16_t AIR_COLOR = WHITE;
const uint16_t UNKNOWN_COLOR = YELLOW;
```

Im Konstruktor der Klasse wird die `ILI9486_SPI` Library initialisiert, danach wird die selbst geschriebene Setup Funktion ausgeführt, in der notwendige Funktionen der Bibliothek ausgeführt werden.

```
tft.init(); // Initialisierung der Library
tft.setSpiKludge(false); // Modus für Treiber ILI9486 (= falscher Treiber) deaktivieren
tft.setRotation(3); // Drehung der Anzeige einstellen
```

4.4.2 Darstellungsfunktionen

Zum Zeichnen auf dem Display werden bereitgestellte Funktionen verwendet wie:

```
void fillScreen(color); // Füllt den Bildschirm
void drawCircle(x_center, y_center, radius, color); // Zeichnet ein Kreis
void drawRect(x_center, y_center, width, height, color); // Zeichnet ein Rechteck
```

4.4.3 Routendarstellung

Die berechnete Route wird nach der Berechnung mit einer erkennbaren Farbe auf der Umgebungsanzeige dargestellt. Dabei werden die Pixel, die die Luft darstellen, mit der Farbe der Route überschrieben.

4.5 Motorensteuerung [Berdan KILIC]

4.5.1 L298N-Treiber

Um die Steuerung erstmals zu testen, werden L298N – Treiber Module verwendet.

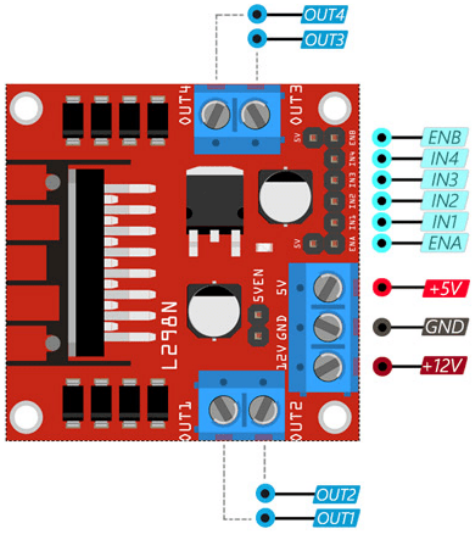


Abbildung 24: L298N-Modul
(lastminuteengineers, 2020)

Der L298N-Chip verfügt über zwei H-Brücken. Jede H-Brücke treibt eine der elektromagnetischen Spulen eines Schrittmotors an und bringt den Motor somit zum Rotieren. Um 4 Motoren anzusteuern werden 2 Module verwendet. (lastminuteengineers, 2020)

Der L298N-Motortreiber wird mit 9V versorgt und ist in der Lage, 2 A Dauerstrom pro Spule zu liefern.

Versorgungspins

Das Motortreibermodul L298N wird über eine 3-polige Schraubklemme mit Strom versorgt.

V_S: Der Pin versorgt die internen H-Brücken des IC mit Strom, um den Motor anzutreiben. An diesen Pin wird eine Eingangsspannung von 9 V angeschlossen.

V_{SS}: wird verwendet, um die Logikschaltung im L298N-IC anzutreiben, welche 5V beträgt.

GND: gemeinsame Erdung. (lastminuteengineers, 2020)

Ausgangspins: Die Ausgangskanäle des L298N-Motortreibers OUT1,OUT2 und OUT3,OUT4 sind zwei Schraubklemmen. An diese Klemmen werden die Motoren angeschlossen.

Kontrollpins: Durch Verwendung der vier Steuerpins IN 1,IN 2,IN3,IN4 wird sowohl die Geschwindigkeit als auch die Drehrichtung des Schrittmotors gesteuert. Diese Pins steuern die Schalter der H-Brückenschaltung im L298N-Chip.

Enable Pins: dienen dazu, den Motor unabhängig von den Eingangssignalen zu aktivieren oder zu deaktivieren. (lastminuteengineers, 2020)

Das Modul wird mit Jumpern auf den Enable Pins geliefert. Wenn die Jumper gesteckt sind, ist der Motor aktiviert. Da der Motor programmgesteuert wird, müssen die Jumper entfernt und diese Pins mit den digitalen Pins des ESP32 verbunden werden. (lastminuteengineers, 2020)

Dadurch können die Pins dafür verwendet werden, um die Geschwindigkeit der mit dem Modul verbundenen Motoren zu steuern. Dies geschieht durch Anwendung der Pulsweitenmodulation[8] (PWM) auf diese Pins. Dabei wird die Breite eines pulsierenden Signals variiert, um die durchschnittliche Leistung, die an den Motor gesendet wird, zu steuern.

[8] https://www.mikrocontroller.net/articles/Pulsweitenmodulation

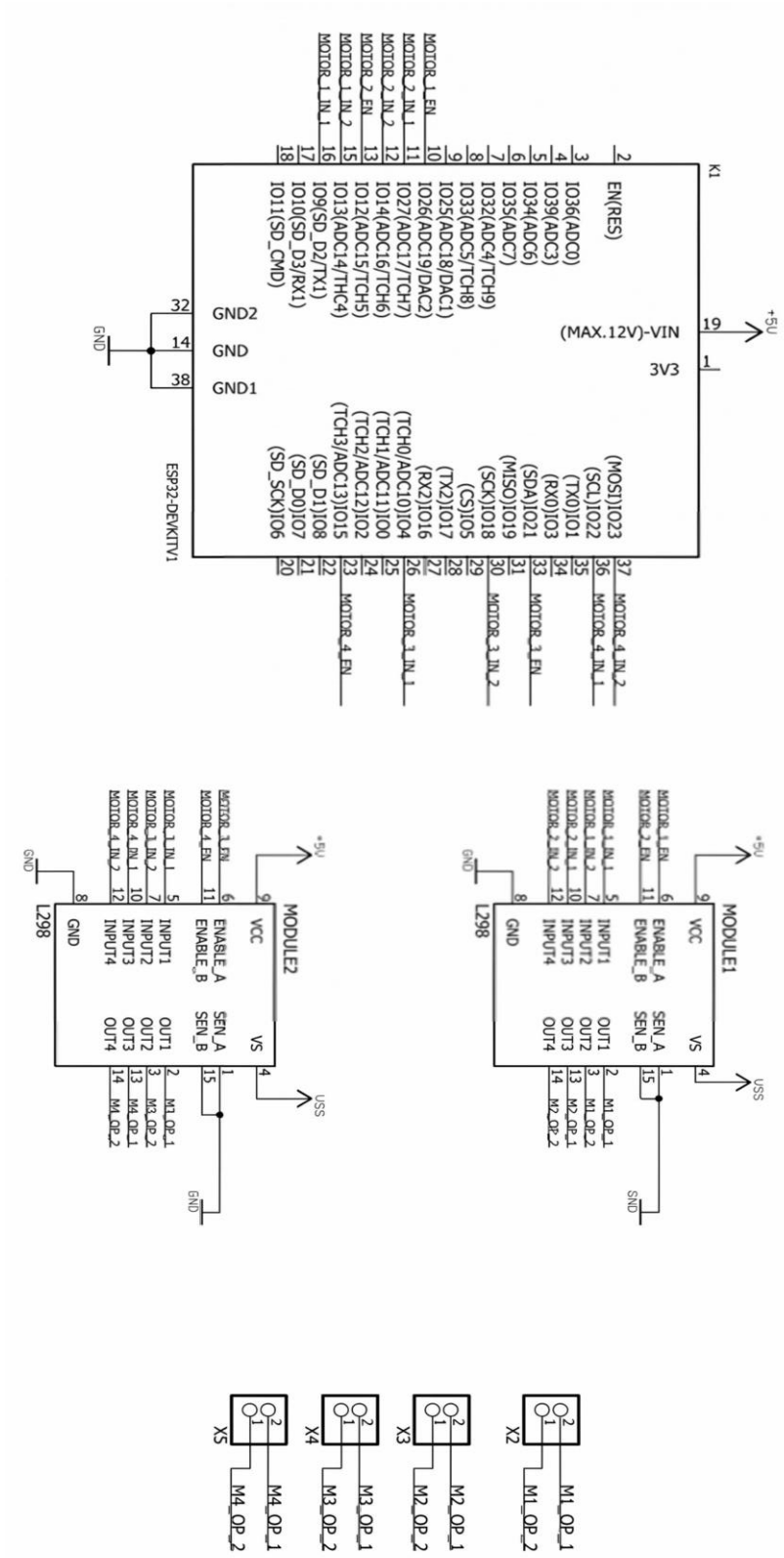


Abbildung 25: Testschaltung mit Modulen

4.5.2 **Code**

Vorerst werden alle Pins des Mikrocontrollers für den Treiber als OUTPUT konfiguriert. Dadurch können logische Signale (High oder Low) erzeugt werden, um die Motoren hinsichtlich ihrer Drehrichtung und Geschwindigkeit präzise steuern zu können.

```
void L298::init() {
  pinMode(driver_1_in1, OUTPUT);
  pinMode(driver_1_in2, OUTPUT);
  pinMode(driver_1_in3, OUTPUT);
  pinMode(driver_1_in4, OUTPUT);
  pinMode(driver_1_en1, OUTPUT);
  pinMode(driver_1_en2, OUTPUT);
  pinMode(driver_2_in1, OUTPUT);
  pinMode(driver_2_in2, OUTPUT);
  pinMode(driver_2_in3, OUTPUT);
  pinMode(driver_2_in4, OUTPUT);
  pinMode(driver_2_en1, OUTPUT);
  pinMode(driver_2_en2, OUTPUT);
}
```

Weiters werden weitere Funktionen für die verschiedenen Fahrtrichtungen erzeugt. (Stopp, Vorwärts, Rückwärts, Rechts, Links)

Funktion für „**Stopp**“

```
void L298::disableAllMotors() {
  digitalWrite(driver_1_in1, LOW);
  digitalWrite(driver_1_in2, LOW);
  digitalWrite(driver_1_in3, LOW);
  digitalWrite(driver_1_in4, LOW);

  digitalWrite(driver_2_in1, LOW);
  digitalWrite(driver_2_in2, LOW);
  digitalWrite(driver_2_in3, LOW);
  digitalWrite(driver_2_in4, LOW);
}
```

Die Drehrichtung der Motoren wird durch das gezielte Setzen der Eingangspins auf HIGH oder LOW bestimmt. Um einen Motor in eine Richtung zu drehen, setzt man einen Eingang auf HIGH und den direkt gegenüberliegenden Eingang auf LOW, was die Polarität der Spannung am Motor und somit seine Drehrichtung bestimmt. Die spezifische Zuordnung von HIGH und LOW zu den Pins hängt von der gewünschten Bewegungsrichtung ab und kann durch experimentelles Testen ermittelt werden.

Funktion für „**Vorwärts**“

```
void L298::forward() {
  digitalWrite(driver_1_in1, HIGH);
  digitalWrite(driver_1_in2, LOW);
  digitalWrite(driver_1_in3, LOW);
  digitalWrite(driver_1_in4, HIGH);

  digitalWrite(driver_2_in1, LOW);
  digitalWrite(driver_2_in2, HIGH);
  digitalWrite(driver_2_in3, HIGH);
  digitalWrite(driver_2_in4, LOW);
}
```

Funktion für „**Rückwärts**“

```
void L298::backward() {
  digitalWrite(driver_1_in1, LOW);
  digitalWrite(driver_1_in2, HIGH);
  digitalWrite(driver_1_in3, HIGH);
  digitalWrite(driver_1_in4, LOW);

  digitalWrite(driver_2_in1, HIGH);
  digitalWrite(driver_2_in2, LOW);
  digitalWrite(driver_2_in3, LOW);
  digitalWrite(driver_2_in4, HIGH);
}
```

Funktion für **„Links“**: beide linken Motoren rückwärts, beide rechten Motoren vorwärts.

```
void L298::left() {
  digitalWrite(driver_1_in1, LOW);
  digitalWrite(driver_1_in2, HIGH);
  digitalWrite(driver_1_in3, LOW);
  digitalWrite(driver_1_in4, HIGH);

  digitalWrite(driver_2_in1, LOW);
  digitalWrite(driver_2_in2, HIGH);
  digitalWrite(driver_2_in3, LOW);
  digitalWrite(driver_2_in4, HIGH);
}
```

Funktion für **„Rechts“**: beide linken Motoren vorwärts, beide rechten Motoren rückwärts.

```
void L298::right() {
  digitalWrite(driver_1_in1, HIGH);
  digitalWrite(driver_1_in2, LOW);
  digitalWrite(driver_1_in3, HIGH);
  digitalWrite(driver_1_in4, LOW);

  digitalWrite(driver_2_in1, HIGH);
  digitalWrite(driver_2_in2, LOW);
  digitalWrite(driver_2_in3, HIGH);
  digitalWrite(driver_2_in4, LOW);
}
```

Schließlich muss die Bewegungsrichtung und Geschwindigkeit des Fahrzeugs basierend auf den Eingabewerten des Joysticks gesteuert werden.

Die gesendeten Joystickwerte werden gespeichert.

```
void carLoop(int joystickX, int joystickY) {
  int X = joystickX;
  int Y = joystickY;
```

Je nach dem wie man den Joystick steuert (vorne, hinten, rechts, links, neutral) werden verschiedene Werte gesendet. Deshalb werden If-Schleifen für die jeweiligen Richtungen programmiert.

Funktion für „**Vorwärts**“

```
if ((X > 1850 && X < 4095) && Y > 1700) {
    int pwm = 0;
    pwm = map(X, 1850, 4095, 0, 255);
    motordrive.forward();
```

Zuallererst wird die PWM-Variable initialisiert.

Die Funktion map() wird verwendet, um einen Wert von einem Bereich in einen anderen umzurechnen. Hier wird die Funktion genutzt, um den Eingabewert X, der sich im Bereich von 1850 bis 4095 bewegt, in einen neuen Bereich von 0 bis 255 zu transformieren.

Die jeweilige Richtungsfunktion, in dem Fall „motordrive.forward()“ wird aufgerufen, um das Fahrzeug in diese Richtung steuern zu können.

```
    if (pwm < 0) {
      pwm = 0;
    }
    if (pwm > 255) {
      pwm = 255;
    }
```

Hiermit wird der PWM-Wert innerhalb eines bestimmten Bereichs begrenzt, nämlich zwischen 0 und 255.

```
   analogWrite(driver_1_en1, pwm);
    analogWrite(driver_1_en2, pwm);
    analogWrite(driver_2_en1, pwm);
    analogWrite(driver_2_en2, pwm);
```

Hier werden die PWM-Werte an die jeweiligen Motoren gesendet, wodurch die Geschwindigkeit angepasst werden kann.

Dieser Vorgang wird für alle Richtungen durchgeführt.

Wenn das Fahrzeug still stehen soll, wird nur die Funktion motordrive.disableAllMotors() aufgerufen und der PWM-Wert von 0 übergeben.

```
if ((X > 1800 && X < 1850) && (Y > 1700 && Y < 1800)) {
    int pwm = 0;
    motordrive.disableAllMotors();
    analogWrite(driver_1_en1, pwm);
    analogWrite(driver_1_en2, pwm);
    analogWrite(driver_2_en1, pwm);
    analogWrite(driver_2_en2, pwm);
  }
```

4.6 Datenerfassung vom Lidar-Sensor [Berdan KILIC]

4.6.1 Sensormessungen

Der Sensor kann 5.000 Messungen pro Sekunde durchführen. Bei jeder Entfernungsmessung sendet der Lidar einen Infrarotlaser aus, der beim Auftreffen auf das Zielobjekt zur Empfangseinheit zurückreflektiert wird. Daraus ergibt sich die Zeit, zu der der Laser ausgesendet und von der Empfangseinheit empfangen wird. Sobald die Entfernungsdaten ermittelt wurden, fusioniert der Sensor die von der Winkelmesseinheit gemessenen Winkelwerte zu Punktwolkendaten und sendet diese. (youyeetoo, 2017)

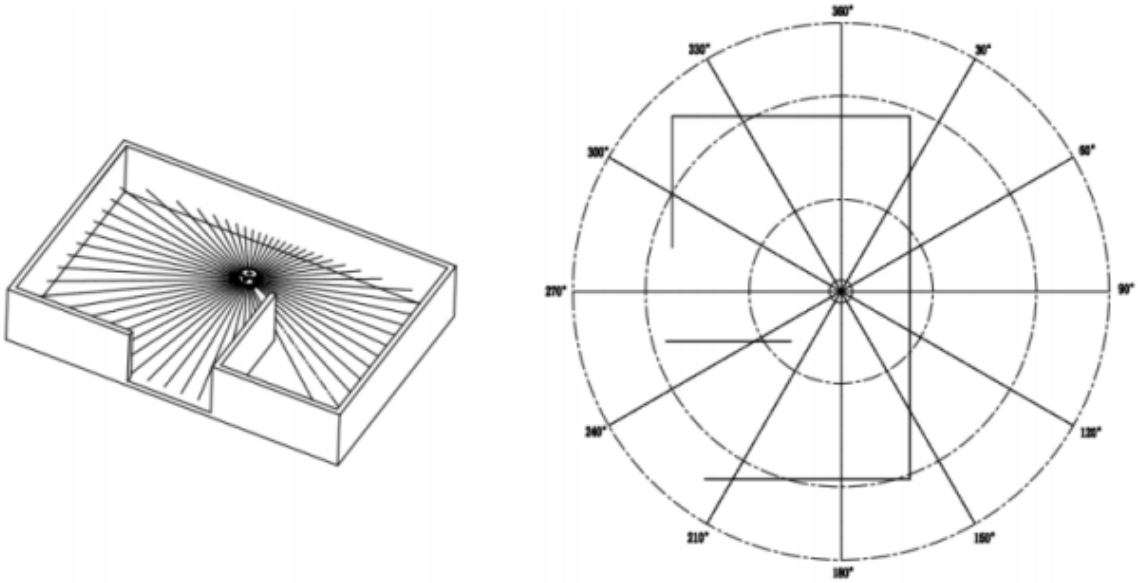


Abbildung 26: Sensor Umgebungsscan (youyeetoo, 2017)

4.6.2 Sensordaten

Der Sensor sendet Datenpakete, die ausgelesen bzw. entwertet werden müssen.

Header	VerLen	Speed		Start angle		Data	End angle		Timestamp		CRC check
54H	1 Byte	LSB	MSB	LSB	MSB	.....	LSB	MSB	LSB	MSB	1 Byte

Abbildung 27: Sensor Datenpaketformat (youyeetoo, 2017)

Header: Länge ->1 Byte, Wert auf 0x54 festgelegt->Beginn des Datenpakets;

VerLen: Länge ->1 Byte, oberen drei Bits geben den Pakettyp an, der auf 1 festgelegt ist, unteren fünf Bits geben die Anzahl der Messpunkte in einem Paket an, die auf 12 festgelegt ist, sodass der Bytewert festgelegt ist bei 0x2C;

Speed: Länge->2 Byte, Einheit ist Grad pro Sekunde und gibt die Geschwindigkeit des Lidar an;

Start angle: Länge->2 Bytes, Einheit beträgt 0,01 Grad-> gibt Startwinkel des Datenpaketpunktes an;

Data: Länge->3 Byte, zeigt Messdaten an;

End angle: Länge->2 Bytes, Einheit beträgt 0,01 Grad-> gibt Endwinkel des Datenpaketpunktes an;

Timestamp: Länge->2 Bytes, Einheit ist ms und Maximum beträgt 30.000;

CRC check: Länge->1 Byte, ergibt sich aus der Überprüfung aller vorherigen Daten; (youyeetoo, 2017)

4.6.3 Code

```
static const uint8_t CrcTable[256] = {
  0x00, 0x4d, 0x9a, 0xd7, 0x79, 0x34, 0xe3, 0xae, 0xf2, 0xbf, 0x68, 0x25,
...
...
...
  0x7f, 0x32, 0xe5, 0xa8};

uint8_t LD19::CalCRC8(uint8_t *p, uint8_t len) {
  uint8_t crc = 0;
  uint16_t i;
  for (i = 0; i < len; i++) {
    crc = CrcTable[(crc ^ *p++) & 0xff];
  }
  return crc;
} (youyeetoo, 2017)
```

Hier wird das Array CrcTable mit 256 Elementen definiert. Die Größe 256 wird gewählt, weil der CRC-Algorithmus[9], der in dieser Implementierung verwendet wird, mit 8-Bit Daten arbeitet. Für jedes mögliche Byte (0-255) gibt es einen vorberechneten CRC-Wert in diesem Array.

Bei der Berechnung des CRC-Wertes für ein Datenpaket wird jedes Byte des Pakets mit dem aktuellen CRC-Wert XOR-verknüpft. Das Ergebnis wird als Index verwendet, um den entsprechenden Wert aus der CrcTable zu holen. Dieser Wert wird dann erneut mit dem CRC XOR-verknüpft, um den neuen CRC zu erhalten. Dieser Vorgang wird für jedes Byte in den Daten wiederholt.

[9] https://mikrocontroller.net/articles/CRC

```
while (Serial.available() > 0) {
    // Serial.println("RX");
    rxByte = Serial.read();
    if (rxByte == 0x54) {
      tmp[count] = rxByte;
      count++;
    } else if (rxByte == 0x2C) {
      tmp[count] = rxByte;
      count++;
```

Die while Schleife läuft, solange Daten über die serielle Schnittstelle verfügbar sind. Es wird immer das nächste verfügbare Byte ausgelesen.

Wenn das gelesene Byte 0x54 ist, wird dies als Start eines neuen Datenpakets interpretiert. Das Byte wird in das Array tmp an der aktuellen Position, die durch count bestimmt wird, gespeichert. Anschließend wird count um 1 erhöht, um für das nächste Byte vorzubereiten.

Wenn das darauffolgende Byte 0x2C ist, bestätigt dies die Struktur des erwarteten Pakets, speziell den Typ und die Anzahl der Messpunkte. Dieses Byte wird ebenfalls in tmp gespeichert, und count wird erneut um 1 erhöht.

```
else if (tmp[0] == 0x54 && tmp[1] == 0x2C) {
      tmp[count] = rxByte;
      if (count >= 46) {
```

Als Erstes wird geprüft, ob die ersten beiden Bytes im Array tmp die Startsignale des Datenpakets sind, nämlich das Header-Byte 0x54 und das VerLen-Byte 0x2C. Dies bestätigt, dass die folgenden Bytes zum aktuellen Datenpaket gehören. Das empfangene Byte wird im tmp Array an der Stelle count gespeichert. Anschließend wird überprüft, ob die Länge des gesammelten Datenpakets, welches 46 Bytes lang ist, erreicht wurde. Dadurch weiß man, dass das Paket vollständig ist.

```
startAngle = (tmp[5] * 256 + tmp[4]) / 100;
endAngle = (tmp[43] * 256 + tmp[42]) / 100;
```

Die Bytes tmp[5] und tmp[4] enthalten die Teile eines 16-Bit-Integer-Wertes, der den Startwinkel darstellt. Dabei ist tmp[5] das höherwertige Byte und tmp[4] das niederwertige Byte. Die Multiplikation von tmp[5] mit 256 verschiebt den Wert nach links, um es als höherwertige Bits zu positionieren. Die Addition von tmp[4] fügt die niederwertigen Bits hinzu. Der resultierende Wert wird durch 100 geteilt, um den Winkel in Grad zu konvertieren. Das selbe wird mit den Bytes tmp[43] und tmp[42] gemacht, da diese den Endwinkel darstellen.

```
mpoints[0] = tmp[7] * 256 + tmp[6];
...
...
...
mpoints[11] = tmp[40] * 256 + tmp[39];
```

Beispiel: tmp[7] und tmp[6] sind zwei aufeinanderfolgende Bytes im Datenpaket, die zusammen einen Entfernungsmesspunkt darstellen.

tmp[7] ist das höherwertige Byte und tmp[6] das niederwertige Byte. tmp[7] * 256 verschiebt das höherwertige Byte um 8 Bits nach links und durch Addition von tmp[6] wird der vollständige 16-Bit-Wert gebildet, wodurch man dann eine Entfernungsmessung hat.

```
crc = CalCRC8((uint8_t *)&tmp, 46);
        if (crc == tmp[46]) {
          for (int i = 0; i < size; i++) {
            arr[i] = mpoints[i];
          }
```

Der CRC-Wert wird für die ersten 46 Bytes des tmp Arrays berechnet und anschließend mit dem im Datenpaket enthaltenen CRC-Wert verglichen. Stimmen diese überein, was auf eine fehlerfreie Übertragung hinweist, werden die validierten Messdaten aus dem mpoints Array in ein anderes Array arr übertragen. Dieser Schritt ist entscheidend, um sicherzustellen, dass nur korrekte und überprüfte Daten für die weitere Verarbeitung und Analyse verwendet werden.

```
step = (endAngle - startAngle) / (12 - 1);
          for (int i = 0; i < aSize; i++) {
            arrAngle[i] = (int)(startAngle + step * i);
            if (arrAngle[i] >= 360) {
              arrAngle[i] -= 360;
            }
          }
```

Als erstes wird die Schrittweite zwischen den einzelnen Messpunkten berechnet. Die Berechnung (endAngle - startAngle) / (12 - 1) teilt die gesamte Winkelspanne zwischen dem Start- und dem Endwinkel gleichmäßig auf die Anzahl der Messpunkte auf.
In dieser Schleife wird jeder Winkel basierend auf dem Startwinkel und der jeweiligen Schrittweite step berechnet und im Array arrAngle gespeichert. Für jeden Punkt wird der Startwinkel um das Produkt aus Schrittweite und Index erhöht, was den Winkel des aktuellen Messpunktes ergibt.

Anschließend wird überprüft, ob der berechnete Winkel 360 Grad überschreitet, was den Umfang eines Kreises darstellt. Ist dies der Fall, wird der Winkel um 360 Grad reduziert, um ihn im gültigen Bereich von 0 bis 359 Grad zu halten.

4.7 Hardwareintegration im Fahrzeug [Berdan KILIC]

4.7.1 Integration des L298N-Treibers

Um eine große Platine mit allen Komponenten des Fahrzeugs zu entwickeln, müssen die L298N-Treiber erstmals aus den Modulen entfernt werden.

Um die Chips in die Platine für die Motorsteuerung einzubauen, müssen Freilaufdioden und Kondensatoren eingebaut werden. Die Freilaufdioden sind wichtig, um beim Schalten hohe Spannungen zu vermeiden. Die Kondensatoren sind an den Versorgungsspannungen eingebaut, um die Störsicherheit zu verbessern. (Stadler, 2023)

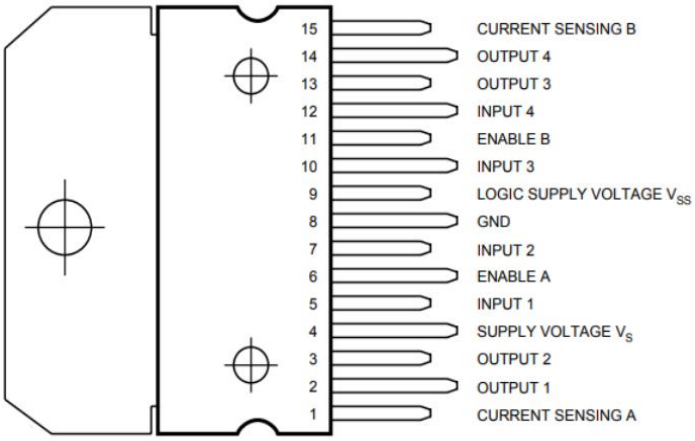


Abbildung 28: Pinbelegung des L298N (Stadler, 2023)

4.7.2 Anschluss eines Gleichstrommotors

„Mit dem L298 können zwei Gleichstrommotoren unabhängig voneinander angesteuert werden. Für den zweiten Motor werden die Eingänge IN 3, IN 4 und Enable B und die Ausgänge OUT 3 und OUT 4 verwendet. Die Motoren hängen an derselben Spannungsversorgung.“ (Stadler, 2023)

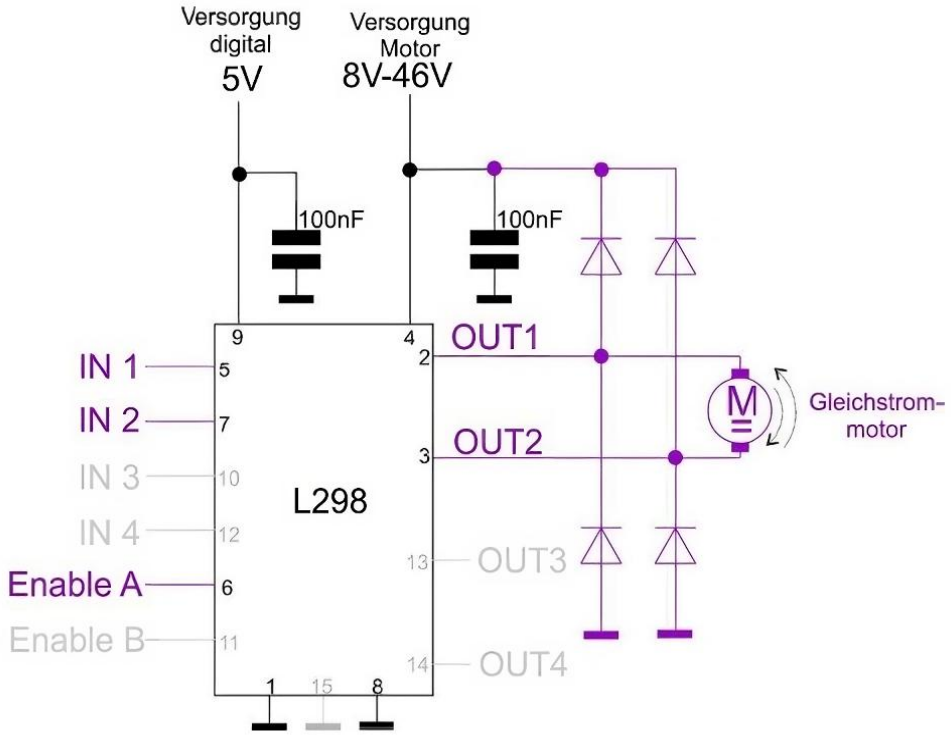


Abbildung 29: Anschluss eines Gleichstrommotors (Stadler, 2023)

4.7.3 Versorgung

Da die Motoren mit 9V und der Sensor, Treiber und ESP32 mit 5V versorgt werden, wird ein Spannungsregler L7805 verwendet. Dadurch müssen nicht separat 9V und 5V geliefert werden, sondern nur 9V, welche dann auf 5V geregelt werden.

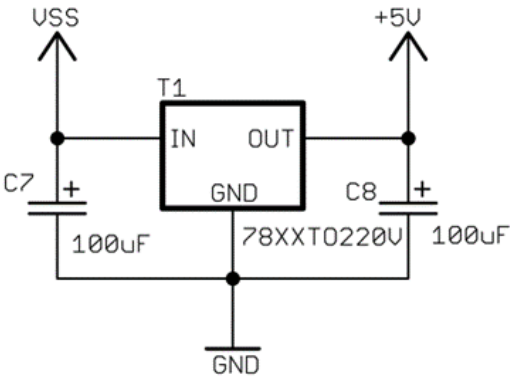


Abbildung 30: Beschaltung des L7805



Abbildung 32: L7805 Spannungsregler (fixpart, 2016)

Für die Versorgung des Fahrzeugs wird ein 9.6V NiMH Akkupack mit 2400mAh und einem Gewicht von 220g verwendet. Bei Betrieb verbraucht das Fahrzeug ungefähr 1.2 Ampere. Das bedeutet, dass der Akku ungefähr 2 Stunden aushält.



Abbildung 31: Fahrzeug-Akku (batteryzone, 2019)

5 Ergebnisse

5.1 Platine der Fernsteuerung [David Kedra]

5.1.1 Stromlaufplan

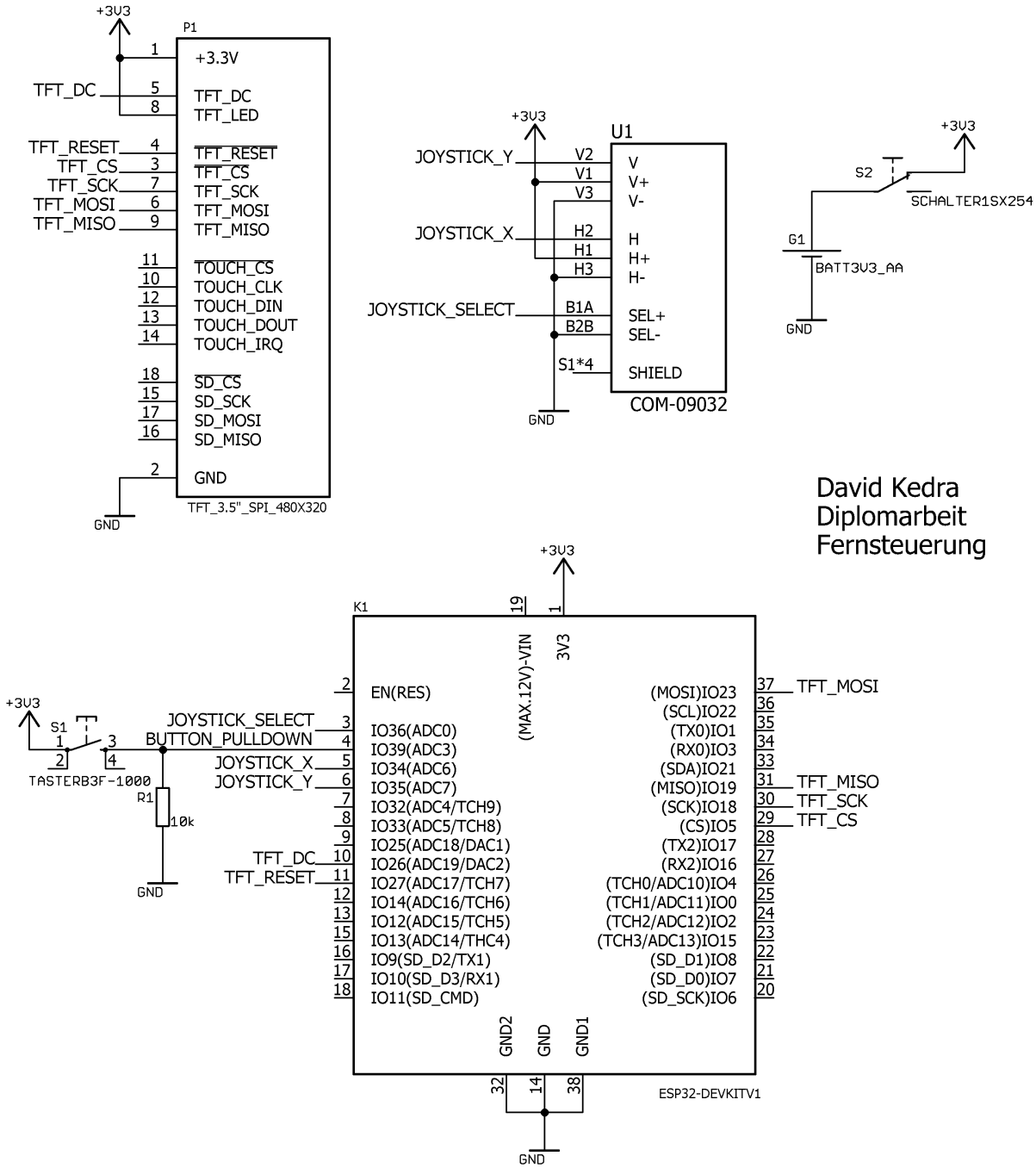


Abbildung 33: Fernsteuerung, Stromlaufplan

5.1.2 Druckvorlage mit GND-Fläche

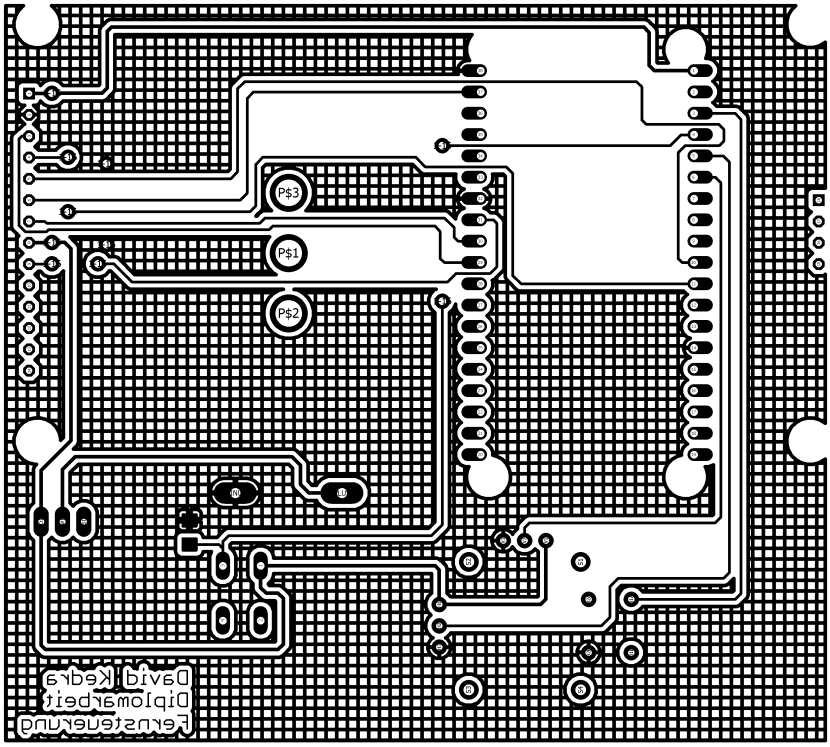


Abbildung 34: Fernsteuerung, Druckvorlage

5.1.3 Druckvorlage ohne GND-Fläche

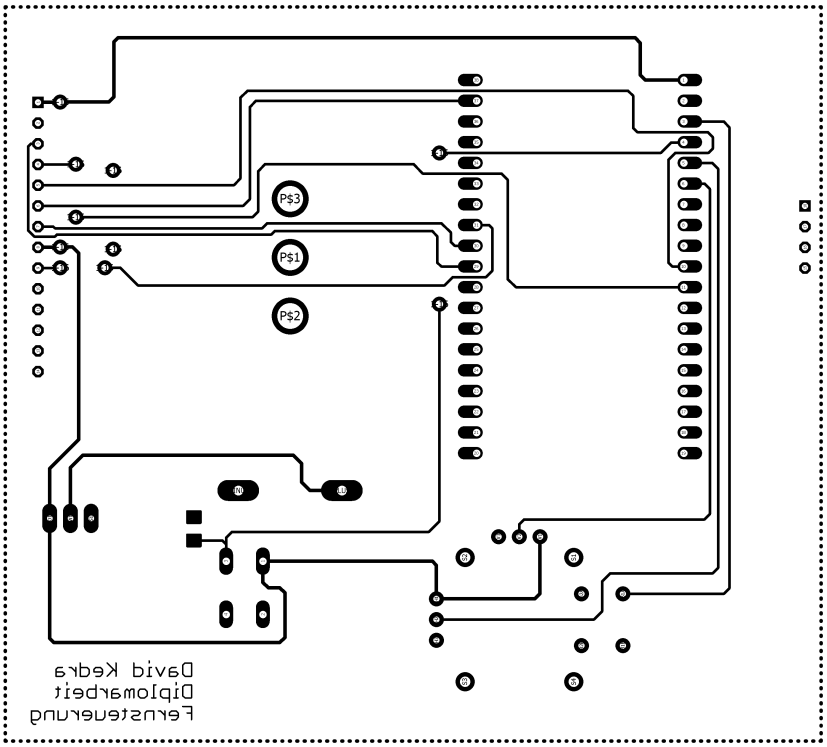


Abbildung 35: Fernsteuerung, Druckvorlage ohne GND-Fläche

5.1.4 Bestückungsplan Top

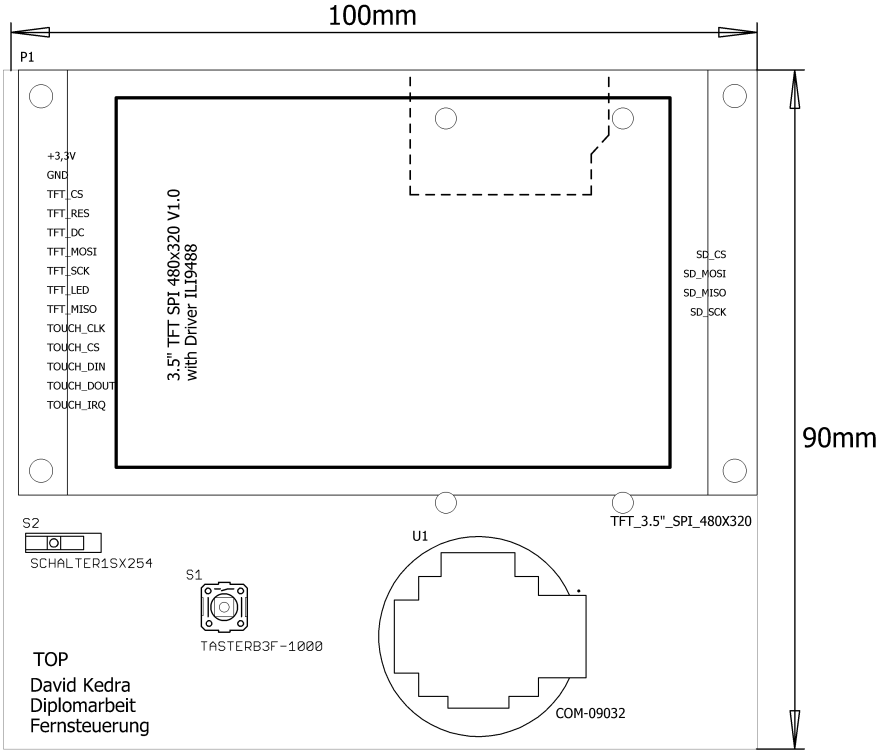


Abbildung 36: Fernsteuerung, Bestückungsplan Top

5.1.5 Bestückungsplan Bottom

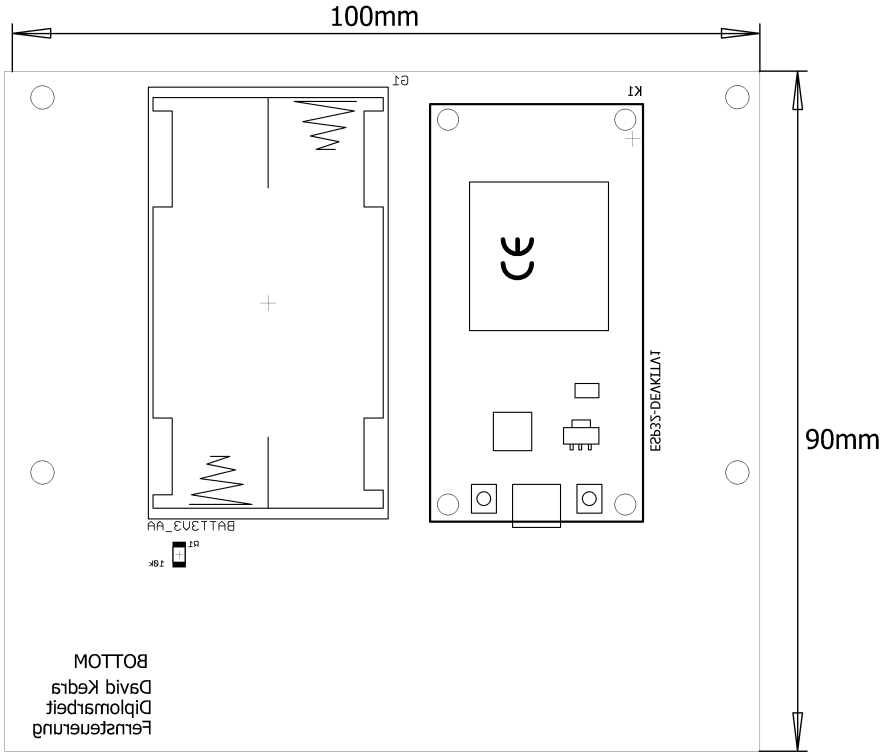
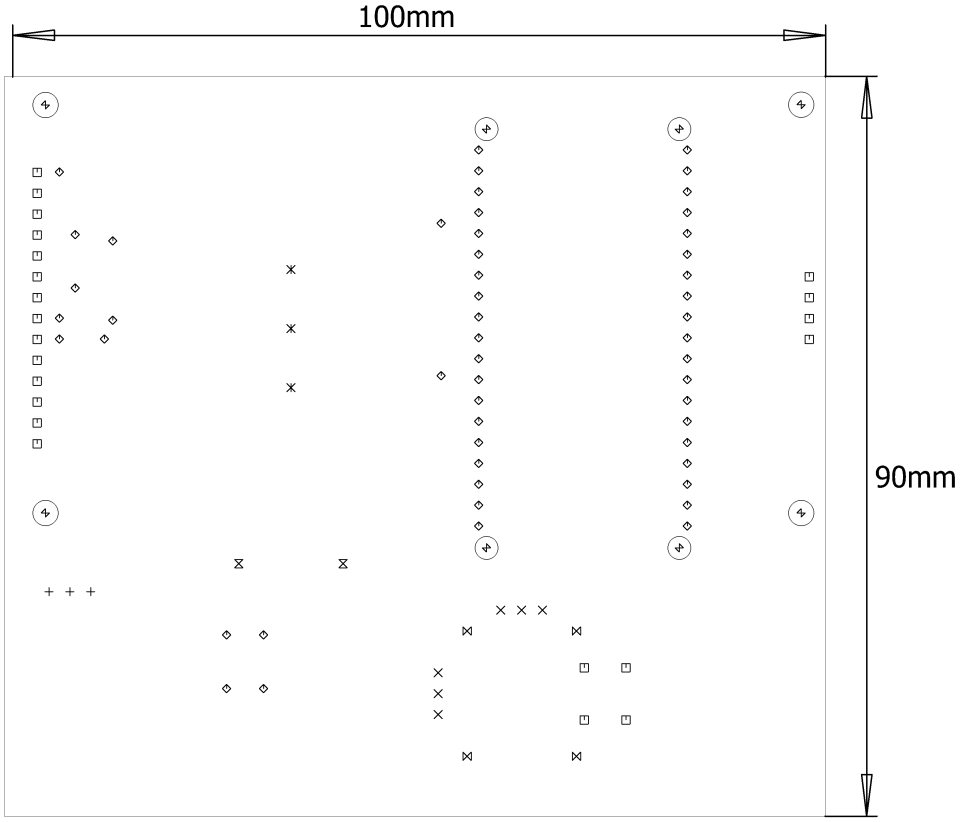


Abbildung 37: Fernsteuerung, Bestückungsplan Bottom

5.1.6 Leiterplattenfertigungszeichnung



LAYER-STACK
01-16
01-20

Sym	N°	Mils	MM	Qty	Plated
+	1	31	0.80	3	YES
×	2	35	0.89	6	YES
◫	3	35	0.90	22	YES
◇	4	39	1.00	52	YES
⧖	5	51	1.30	2	YES
⋈	6	55	1.40	4	YES
⏚	7	110	2.80	4	NOT
ϟ	8	122	3.10	4	NOT
⊛	9	130	3.30	3	YES

Abbildung 38: Fernsteuerung, Leiterplattenfertigungszeichnung

5.1.7 I/O-Belegung des ESP32 der Fernsteuerung [David Kedra]

Nr.	GPIO-Nr.	Zweck	Erklärung
1	5	Display: CS	Chipauswahl
2	18	Display: SCK	serieller Takt
3	19	Display: MISO	serieller Dateneingang
4	23	Display: MOSI	serieller Datenausgang
5	26	Display: DC	Registerauswahl
6	27	Display: RST	rücksetzen
7	34	Joystick: X	X-Achse
8	35	Joystick: Y	Y-Achse
9	36	Joystick: Select	Joystick-Taster Betätigung
10	39	Taster	Betätigung im Pulldown-Betrieb

Tabelle 7: I/O-Belegung Fernsteuerung

5.1.8 Stückliste (Fernsteuerung) [David Kedra]

Nr.	Stückzahl	Wert	Bauform	Beschreibung
1	1	10k	R1206	Widerstand
2	1	BATT3V3_AA	BATT3V3_AA	Batterieanschluss
3	1	COM-09032	COM-09032	Joystick
4	1	ESP32	ESP32DEVKITV11000MIL	ESP32 Mikrocontroller
5	1	SCHALTER1SX254	SCHALTER1SX254	Schalter
6	1	TASTERB3F-1000	TASTERB3F-1000	Drucktaster
7	1	TFT 3.5 SPI 480X320	TFT 3.5 SPI 480X320	Display

Tabelle 8: Stückliste (Fernsteuerung)

5.2 Platine des Fahrzeugs [Berdan KILIC]

5.2.1 Stromlaufplan

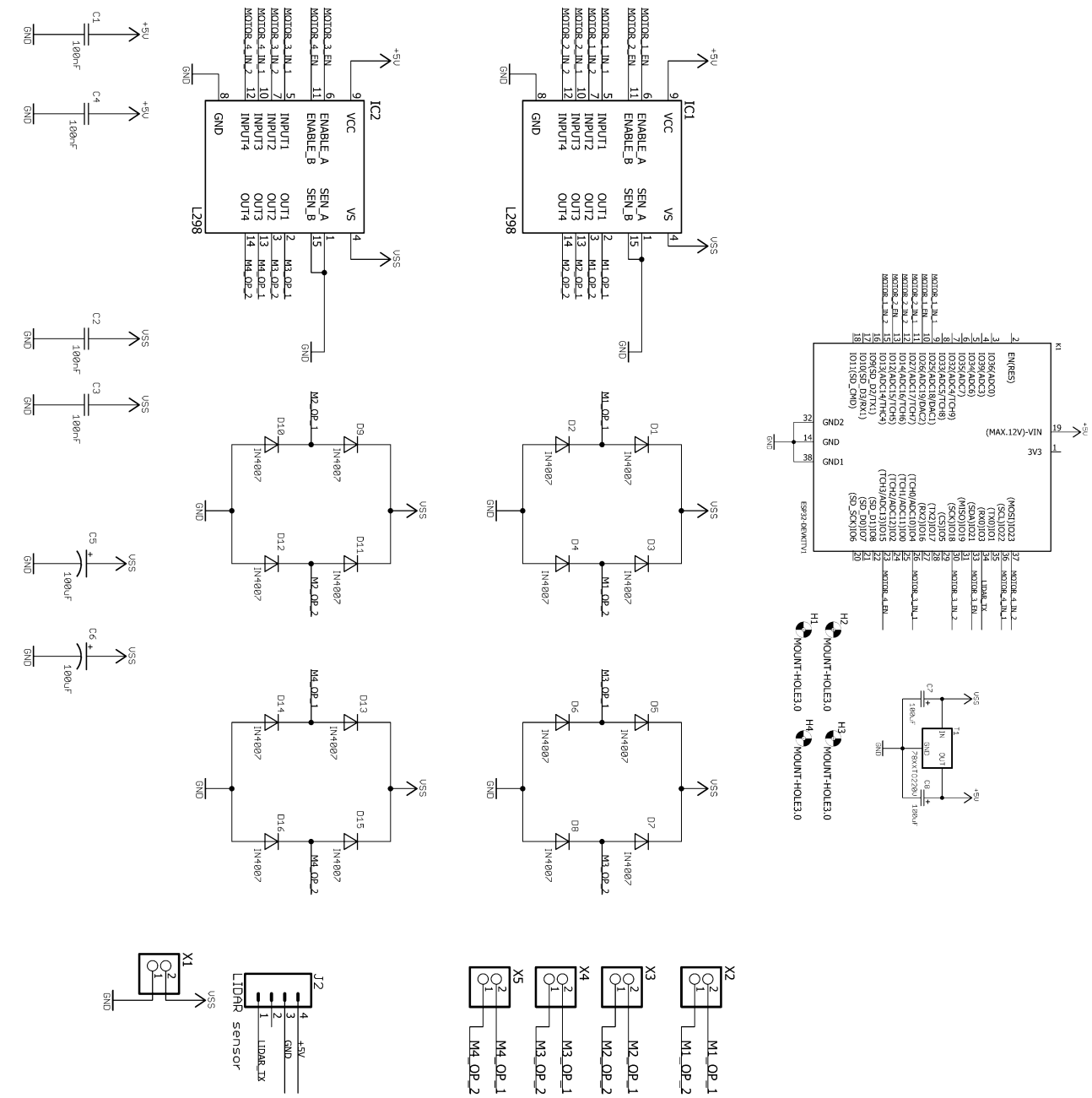


Abbildung 39: Stromlaufplan (Fahrzeug)

5.2.2 Druckvorlage mit GND-Fläche

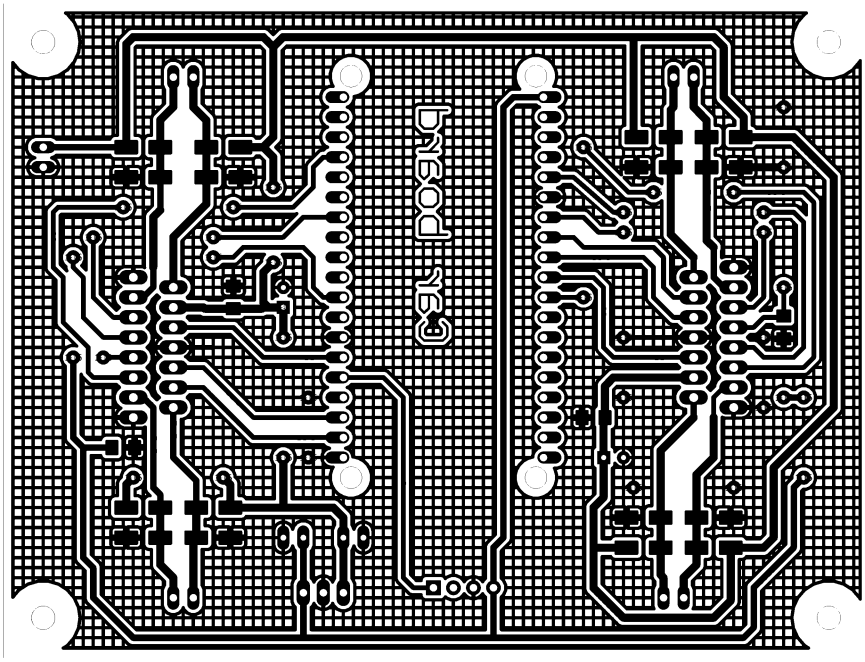


Abbildung 40: Druckvorlage mit GND-Fläche (Fahrzeug)

5.2.3 Druckvorlage ohne GND-Fläche

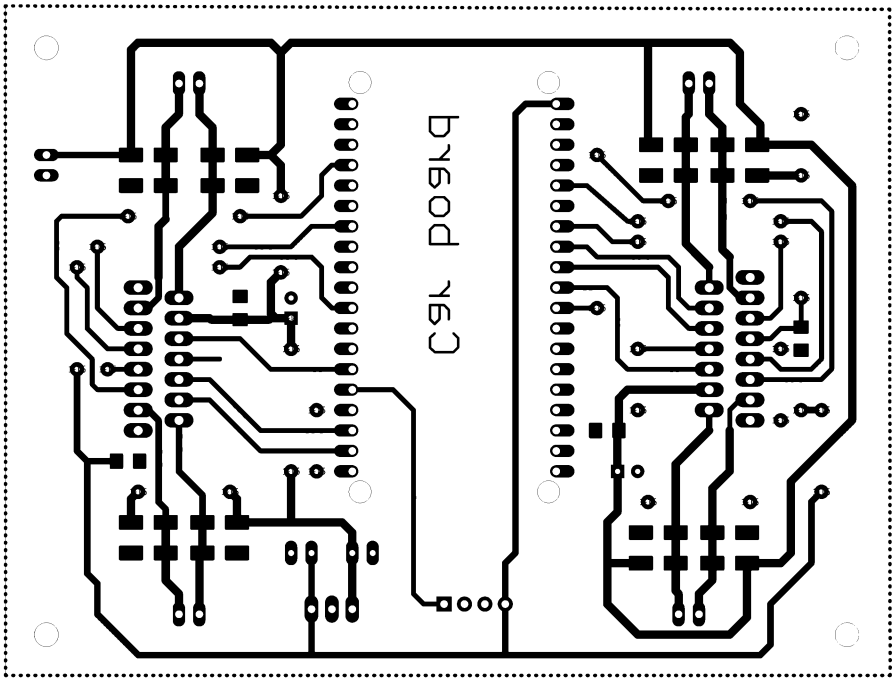


Abbildung 41: Druckvorlage ohne GND-Fläche (Fahrzeug)

5.2.4 Bestückungsplan (Top)

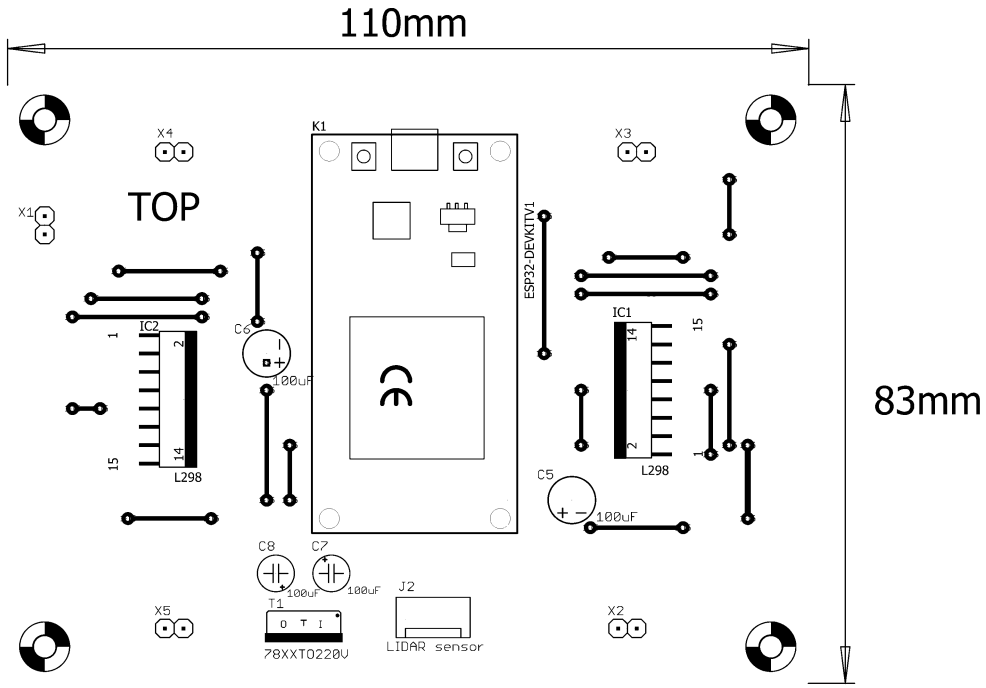


Abbildung 42: Bestückungsplan-Top (Fahrzeug)

5.2.5 Bestückungsplan (Bottom)

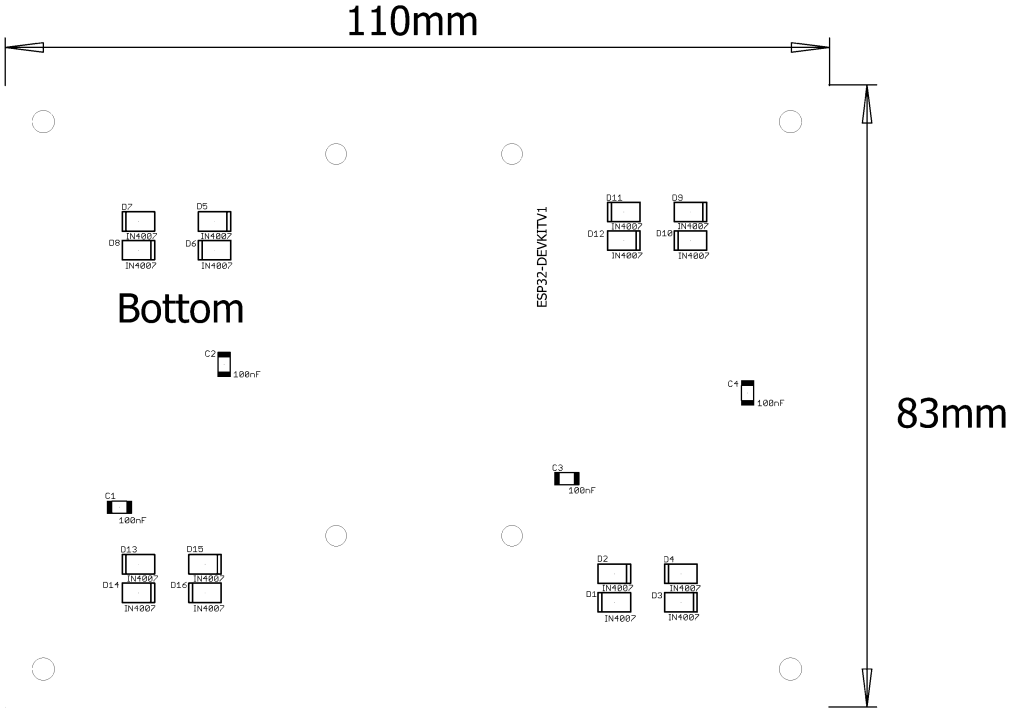
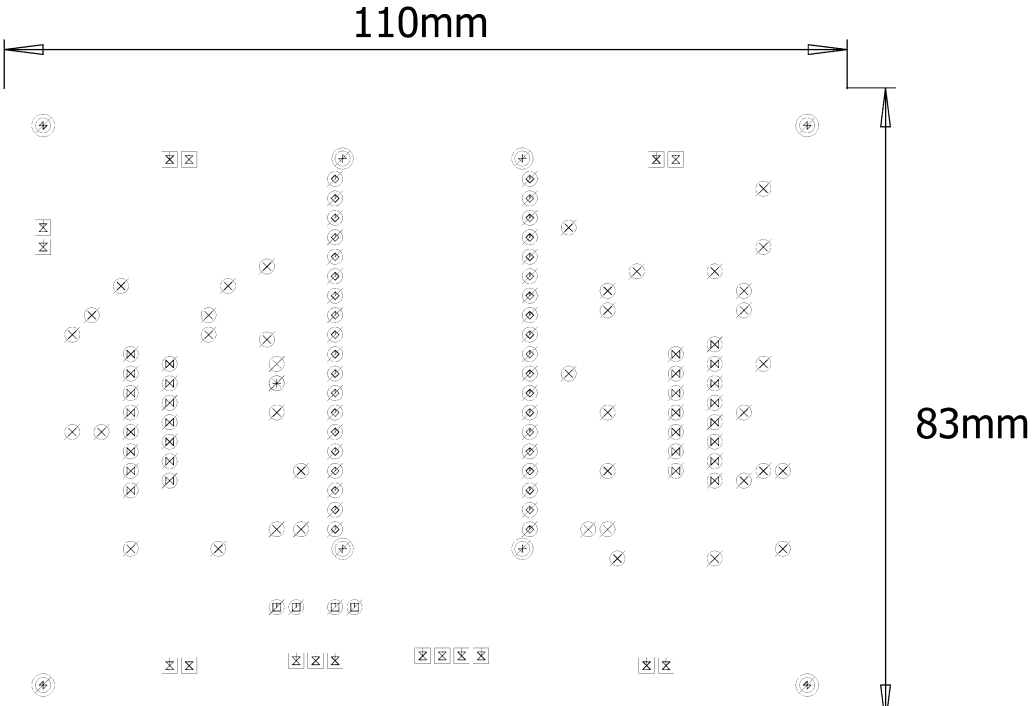


Abbildung 43: Bestückungsplan-Bottom (Fahrzeug)

5.2.6 Leiterplattenfertigungszeichnung



LAYER-STACK
01-16
01-20

Sym	N°	Mils	MM	Qty	Plated
+	1	24	0.60	1	YES
×	2	28	0.70	40	YES
□	3	31	0.80	4	YES
◇	4	39	1.00	38	YES
⧖	5	40	1.02	17	YES
⋈	6	47	1.20	30	YES
✣	7	110	2.80	4	NOT
ϟ	8	118	3.00	4	NOT

Abbildung 44: Leiterplattenfertigungszeichnung (Fahrzeug)

5.2.7 I/O-Belegung des ESP32 des Fahrzeugs

Nr.	GPIO-Nr.	Zweck	Erklärung
1	25,13	MOTOR_1_IN	2 Motoreingänge um den ersten Motor zu steuern
2	26	MOTOR_1_EN	1 Enable-Eingang um den ersten Motor ein/auszuschalten
3	27,14	MOTOR_2_IN	2 Motoreingänge um den zweiten Motor zu steuern
4	12	MOTOR_2_EN	1 Enable-Eingang um den zweiten Motor ein/auszuschalten
5	4,18	MOTOR_3_IN	2 Motoreingänge um den dritten Motor zu steuern
6	21	MOTOR_3_EN	1 Enable-Eingang um den dritten Motor ein/auszuschalten
7	22,23	MOTOR_4_IN	2 Motoreingänge um den vierten Motor zu steuern
8	15	MOTOR_4_EN	1 Enable-Eingang um den vierten Motor ein/auszuschalten
9	3	LIDAR_TX	Datenleitung des Sensors

Tabelle 9: I/O-Belegung (Fahrzeug)

5.2.8 Stückliste (Fahrzeug)

Stückzahl	Wert	Bauform	Beschreibung
5		PINHD-1X2	PIN HEADER
4	100nF	C1206	Kondensatoren
2	100uF	100UF-POLAR-RADIAL-2.5MM-25V-20%	100µF polarized capacitors
2	100uF	ELKORM2,5-DM5	ELKO
1	78XXTO220V	78XXTO220V	Positive Spannungsregler der Serie 78xx
16	IN4007	DIODEDO214AC	Diode
2	L298	L298	DUAL FULL-BRIDGE DRIVER

Tabelle 10: Stückliste (Fahrzeug)

5.3 Flussdiagramm Fernsteuerung [David Kedra]

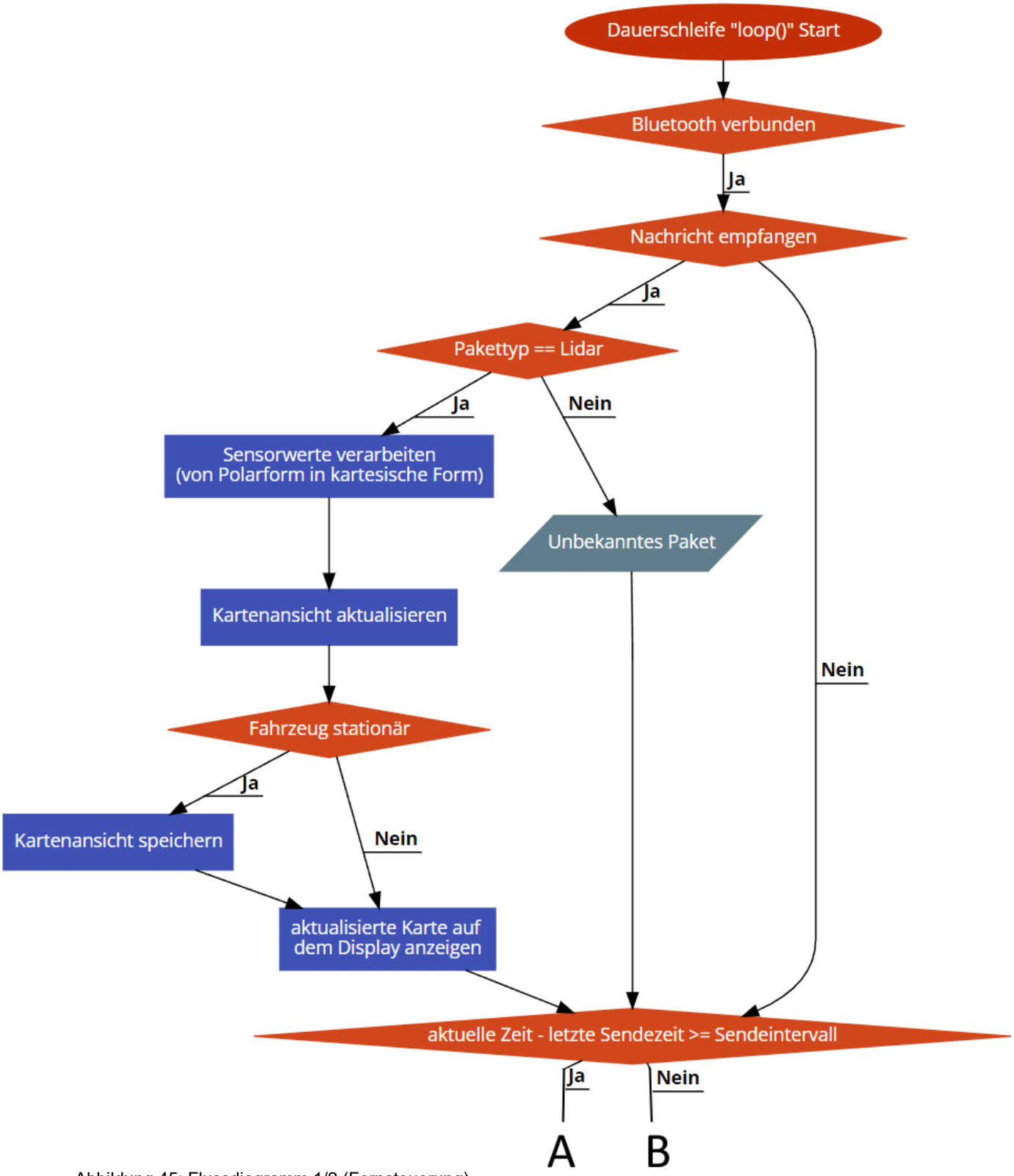


Abbildung 45: Flussdiagramm 1/2 (Fernsteuerung)

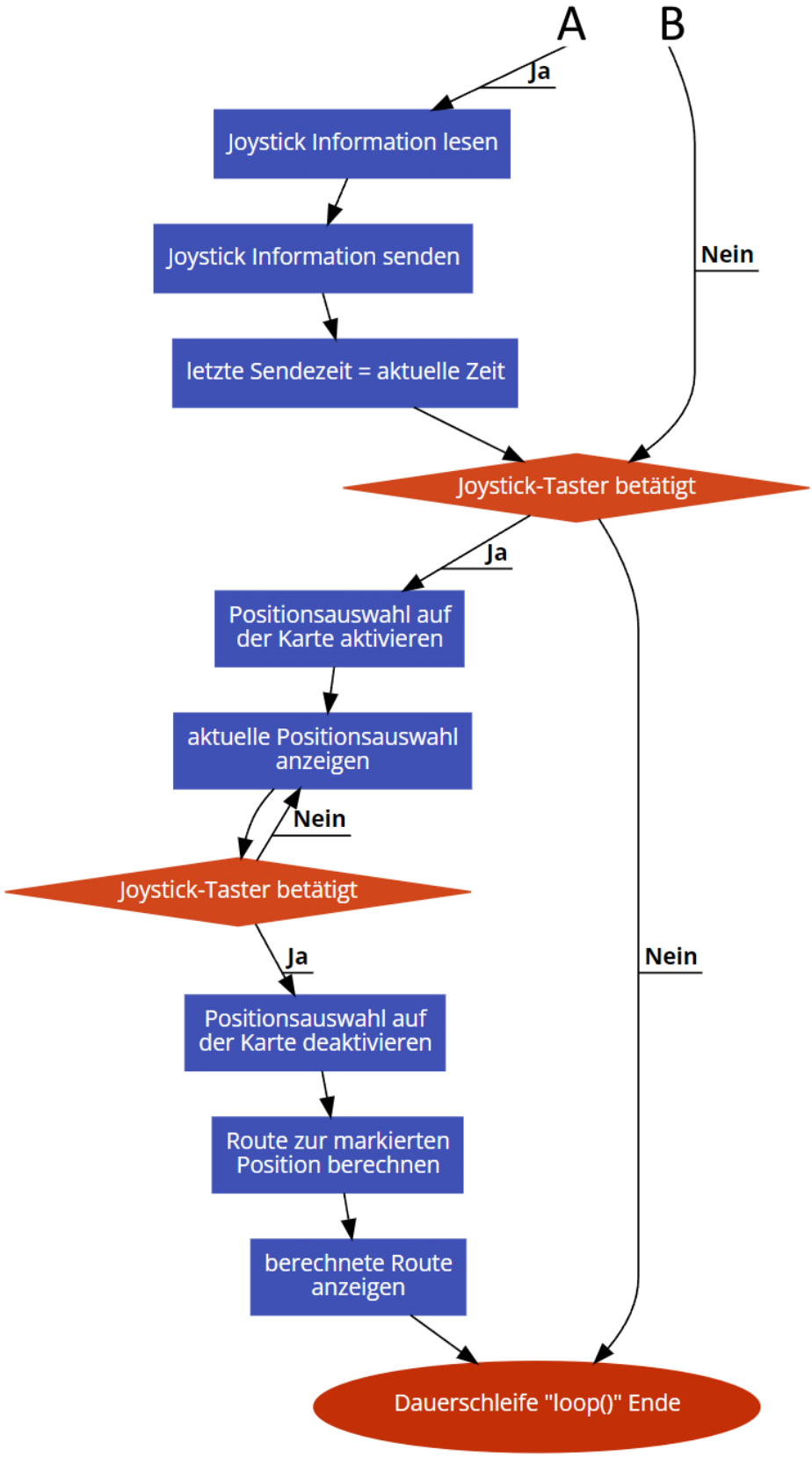


Abbildung 46: Flussdiagramm 2/2 (Fernsteuerung)

5.4 Flussdiagramm Fahrzeug [Berdan KILIC]

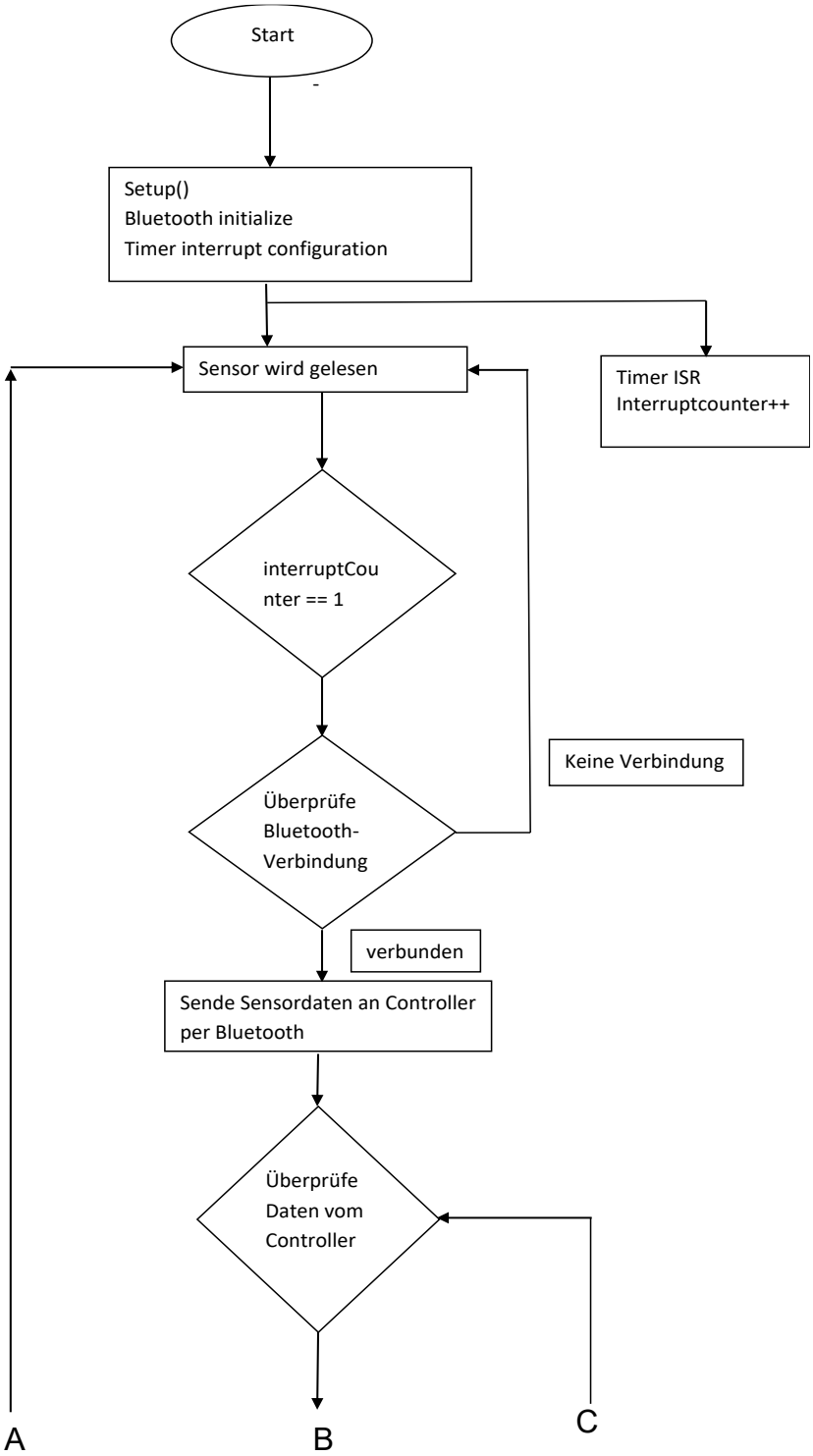


Abbildung 47: Flussdiagramm 1/3 (Fahrzeug)

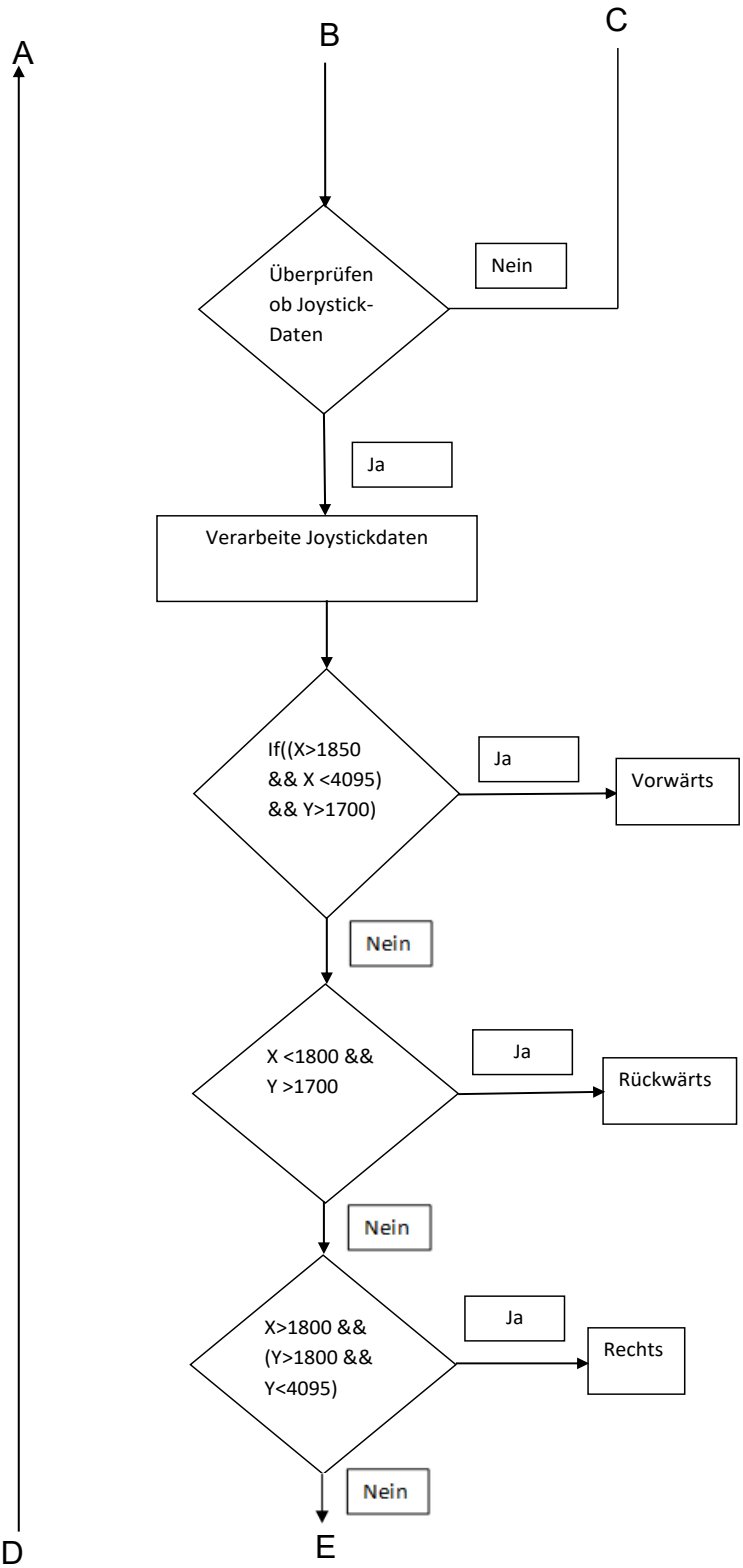


Abbildung 48: Flussdiagramm 2/3 (Fahrzeug)

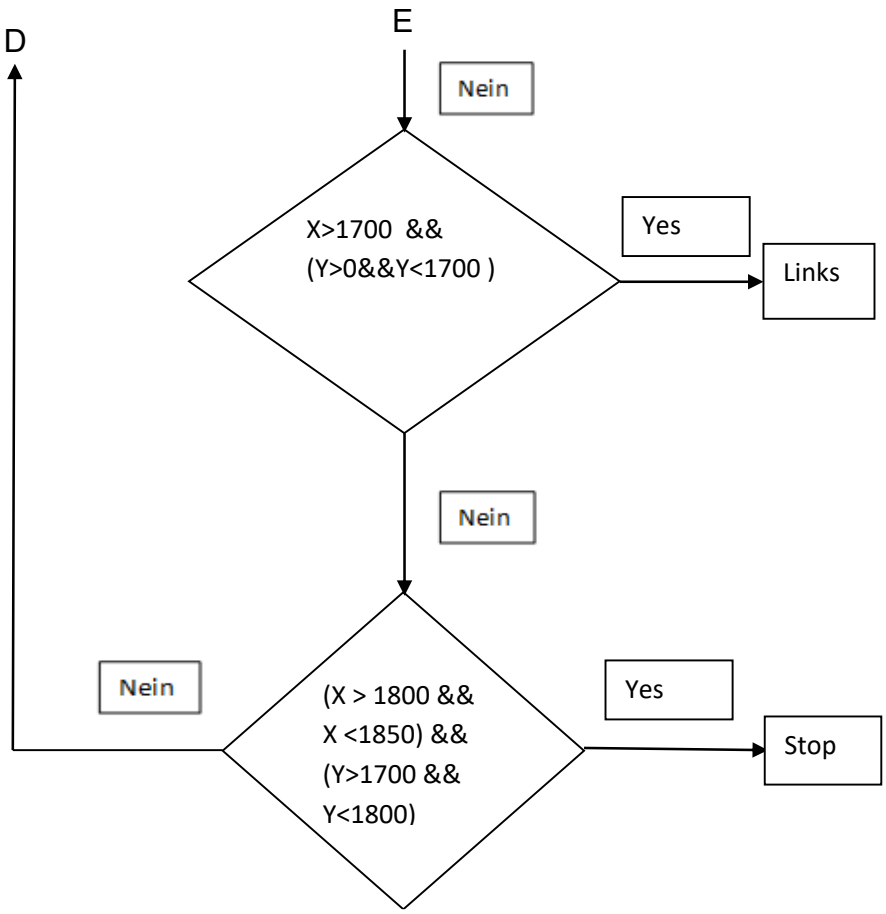


Abbildung 49: Flussdiagramm 3/3 (Fahrzeug)

6 Literatur- und Quellenverzeichnis

aliexpress. (2017). Von https://de.aliexpress.com/i/1005005657168677.html abgerufen

batteryzone. (2019). Von https://batteryzone.de/products/9-6v-nimh-akku-2400mah-wiederaufladbarer-aa-akkupack-mit-usb-ladekabel-und-ket-2p-anschluss?variant=39782695698640&currency=EUR&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic&gad_ abgerufen

digikey. (4 2024). Von https://mm.digikey.com/Volume0/opasdata/d220001/medias/images/2540/COM-09032.jpg abgerufen

Eagle "joystick COM-09032.lbr". (4 2024). Von snapeda: https://www.snapeda.com/parts/COM-09032/SparkFun%20Electronics/view-part abgerufen

ebay. (4 2024). Von https://i.ebayimg.com/images/g/nREAAOSw~Jlh-4ll/s-l500.jpg abgerufen

fixpart. (2016). Von https://fixpart.at/produkt/stmicroelectronics-l7805acv-positiver-spannungsstabilisator abgerufen

getgoods. (2022). Von https://www.getgoods.com/at-de/products/567917/Joy-it-com-Motor01-Getriebemotor-Gelb-Schwarz-Passend-fuer-Einplatinen-Computer-Arduino-Banana-P.html abgerufen

heise. (Juni 2016). Von https://www.heise.de/select/make/2016/6/1482398401198797 abgerufen

lastminuteengineers. (2020). Von https://lastminuteengineers.com/stepper-motor-l298n-arduino-tutorial/?utm_content=cmp-true#google_vignette abgerufen

lcdwiki. (4 2024). Von http://www.lcdwiki.com/3.5inch_SPI_Module_ILI9488_SKU:MSP3520 abgerufen

mathe-mit-methode. (4 2024). Von http://www.mathe-mit-methode.com/schlaufuchs_web/elektrotechnik/mikrocontroller_lernmaterial/microcontroller_allgemein/mikrocontroller_ext_hardware/mikrocontroller_uart.html abgerufen

Stadler, K. (8. März 2023). 02_03_Synchronmaschine. Österreich.

thinkrobotics. (4 2024). Von https://thinkrobotics.com/cdn/shop/products/HC-05-Wireless-Bluetooth_720x.png abgerufen

Wikipedia. (4 2024). Von https://de.wikipedia.org/wiki/Simultaneous_Localization_and_Mapping abgerufen

Wikipedia. (4 2024). Von https://de.wikipedia.org/wiki/Odometrie abgerufen

youyeetoo. (2017). Von https://wiki.youyeetoo.com/en/Lidar/D300 abgerufen

7 Abbildungsverzeichnis

Abbildung 1: Blockschaltbild 11
Abbildung 2: Fernsteuerung Schaubild 12
Abbildung 3: am Bildschirm geplante Visualisierung 12
Abbildung 4: ESP32-WROOM-32 Fernsteuerung, (ebay, 2024) 13
Abbildung 5: Bildschirm, (lcdwiki, 2024) 14
Abbildung 6: Joystick, (digikey, 2024) 17
Abbildung 7: HC-05, (thinkrobotics, 2024) 18
Abbildung 8: ESP32 für Fahrzeug (ebay, 2024) 19
Abbildung 9: L298N-Chip (Stadler, 2023) 20
Abbildung 10: Verwendeter Schrittmotor (getgoods, 2022) 21
Abbildung 11: Ersatzschaltbild eines Schrittmotors (heise, 2016) 21
Abbildung 12: Messschaltung für die Strom/Spannungskennlinie über der Drehzahl 22
Abbildung 13: Stromkennlinie über Drehzahl 22
Abbildung 14: Spannungskennlinie über Drehzahl 22
Abbildung 15: Lidar-Sensor (aliexpress, 2017) 24
Abbildung 16: Testaufbau Bluetooth 25
Abbildung 17: Bluetooth Datenformatierung 28
Abbildung 18: Bluetooth Datenformatierung Joystick 29
Abbildung 19: Bluetooth Datenformatierung Lidar 29
Abbildung 20: Lee-Alg., Ausgangslage 45
Abbildung 21: Lee-Alg., Kalkulation 45
Abbildung 22: Lee-Alg., Ergebnis 45
Abbildung 23, Berechnete Route zur markierten Position 46
Abbildung 24: L298N-Modul (lastminuteengineers, 2020) 50
Abbildung 25: Testschaltung mit Modulen 52
Abbildung 26: Sensor Umgebungsscan (youyeetoo, 2017) 58
Abbildung 27: Sensor Datenpaketformat (youyeetoo, 2017) 59
Abbildung 28: Pinbelegung des L298N (Stadler, 2023) 64
Abbildung 29: Anschluss eines Gleichstrommotors (Stadler, 2023) 65
Abbildung 30: Beschaltung des L7805 66
Abbildung 31: Fahrzeug-Akku (batteryzone, 2019) 66
Abbildung 32: L7805 Spannungsregler (fixpart, 2016) 66

Abbildung 33: Fernsteuerung, Stromlaufplan 67
Abbildung 34: Fernsteuerung, Druckvorlage 68
Abbildung 35: Fernsteuerung, Druckvorlage ohne GND-Fläche 68
Abbildung 36: Fernsteuerung, Bestückungsplan Top 69
Abbildung 37: Fernsteuerung, Bestückungsplan Bottom 69
Abbildung 38: Fernsteuerung, Leiterplattenfertigungszeichnung 70
Abbildung 39: Stromlaufplan (Fahrzeug) 72
Abbildung 40: Druckvorlage mit GND-Fläche (Fahrzeug) 73
Abbildung 41: Druckvorlage ohne GND-Fläche (Fahrzeug) 73
Abbildung 42: Bestückungsplan-Top (Fahrzeug) 74
Abbildung 43: Bestückungsplan-Bottom (Fahrzeug) 74
Abbildung 44: Leiterplattenfertigungszeichnung (Fahrzeug) 75
Abbildung 45: Flussdiagramm 1/2 (Fernsteuerung) 77
Abbildung 46: Flussdiagramm 2/2 (Fernsteuerung) 78
Abbildung 47: Flussdiagramm 1/3 (Fahrzeug) 79
Abbildung 48: Flussdiagramm 2/3 (Fahrzeug) 80
Abbildung 49: Flussdiagramm 3/3 (Fahrzeug) 81

8 Tabellenverzeichnis

Tabelle 1: Arbeitspakete David Kedra 9
Tabelle 2: Arbeitspakete Berdan Kilic 10
Tabelle 3: Bildschirm Parameter, (lcdwiki, 2024) 14
Tabelle 4: Bildschirm Schnittstellen, (lcdwiki, 2024) 15
Tabelle 5: PIN-Tabelle für L298N (Stadler, 2023) 20
Tabelle 6: Sensor-Pinbelegung (youyeetoo, 2017) 24
Tabelle 7: I/O-Belegung Fernsteuerung 71
Tabelle 8: Stückliste (Fernsteuerung) 71
Tabelle 9: I/O-Belegung (Fahrzeug) 76
Tabelle 10: Stückliste (Fahrzeug) 76
Tabelle 11: Stückliste 87
Tabelle 12: Kostendarstellung 87

9 Verwendete Abkürzungen

HTL	Höhere Technische Lehranstalt	DC	Direct Current
UART	Universal Asynchronous Receiver / Transmitter	PWM	Pulsweitenmodulation
IC	Integrated Circuit	RAM	Random Access Memory
GND	Ground	TTL	Transistor-Transistor-Logic
IO, I/O	Input / Output	LD	Lidar Detector
TFT	Thin Film Transistor	CRC	Cycling Redundancy Check
LCD	Liquid Crystal Display	NiMH	Nickel-Metallhydrid

10 Anhang

10.1 Stückliste

Nr.	Anzahl	Name	Anwendung
1.	1	TFT-Bildschirm	Anzeige der Fernsteuerung
2.	1	360° Lidar Sensor	360° Entfernungsmessung der Umgebung des Fahrzeugs
3.	2	H-Brücke Motortreiber L298N-Modul	Treibermodule zur Motorsteuerung
4.	1	Fahrgestell	Mechanisches Gestell zur Befestigung der Komponenten
5.	4	DC-Motor	Motoren zum Antrieb des Fahrzeugs
6.	2	Mikrocontroller ESP32-WROOM-32	programmierbarer Mikrocontroller
7.	1	Joystick COM-09032	Fahrzeugsteuerung
8.	1	Taster	Benutzereingabe
9.	2	Schalter	Ein-/Ausschalter

Tabelle 11: Stückliste

10.2 Kostendarstellung

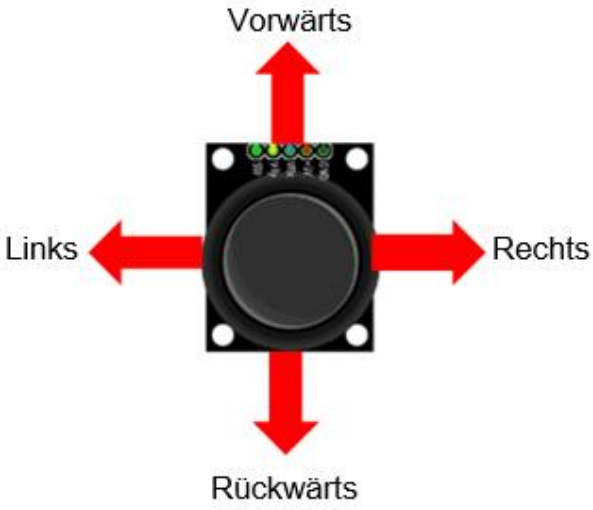
Nr.	Anzahl	Name	Preis	Link
1.	1	TFT-Bildschirm	22,96 €	https://www.amazon.de/dp/B0BWQYVYFL
2.	1	360° Lidar Sensor	61,14 €	https://www.amazon.de/dp/B0B1QCV4XR
3.	1	(2 Stück) H-Brücke Motortreiber L298N-Modul	9,88 €	https://www.amazon.de/dp/B08MZZTXG6
4.	1	Fahrgestell inklusive 4 Motoren	22,80 €	https://www.getgoods.com/at-de/products/567916
5.	1	(2 Stück) ESP32-WROOM-32	13,10 €	https://www.amazon.de/dp/B0C61DT5YX
6.	1	(10 Stück) Ein-/Ausschalter	1,43 €	https://www.temu.com/at/10pcs-snap-in-boat-rocker-switch-toggle-power-spst-on-off-2-pin-ac-250v-6a-125v-10a-on-off-switch-2-pin-position-snap-switch-for-car-auto-home-appliances-g-601099539711238.html

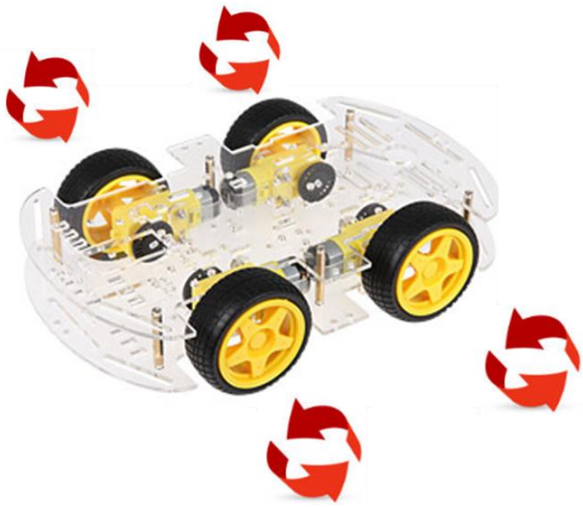
Tabelle 12: Kostendarstellung

10.3 Bedienungsanleitung

Das Fahrzeug und die Fernbedienung können über einen Schalter eingeschaltet werden. Die Bluetooth-Kopplung erfolgt automatisch, wobei dies wenige Sekunden dauern kann.

War die Kopplung erfolgreich, wird die Umgebung des Fahrzeugs auf dem Display der Fernsteuerung angezeigt und das Fahrzeug kann gesteuert werden. Um das Fahrzeug beispielsweise nach vorne zu bewegen, muss der Joystick genau nach vorne bewegt werden. Je weiter der Joystick bewegt wird, desto mehr nimmt das Fahrzeug an Geschwindigkeit zu.





Durch Betätigung des Joysticks wird das Fahren unterbrochen und eine Auswahl auf dem Display aktiviert. Mittels Joysticks kann eine Position auf der Karte ausgewählt werden. Die schnellstmögliche Route zu dieser ausgewählten Position wird berechnet und auf dem Bildschirm angezeigt.